UNIVERSITE DU BURUNDI

INSTITUT DE STATISTIQUE APPLIQUEE "ISTA"

Informatique décisionnelle: Master I
Cours d'Optimisation Combinatoire

Vianney MBAZUMUTIMA (PhD)

April 4, 2025

ISTA

Contents

1 Quelques notions de rappel **2**

1.1 Programmation linéaire 2

1.2 Quelques rappels 2

1.2.1 Ecriture matricielle d'un système d'équations du premier degré à plusieurs inconnues 2

1.2.2 Rang d'une matrice 4

1.2.3 Base d'une matrice 6

1.3 Rappel de géométrie 8

1.4 Formulation générale d'un problème de programmation linéaire "PPL" 9

1.5 Exemples de programme linéaire 11

1.5.1 Le problème de la nourriture pour chiens 11

1.5.2 Gestion d'une brasserie 13

1.6 Exercices 14

1.7 Méthodes de résolution en programmation linéaire 15

1.7.1 Méthode graphique 15

1.7.2 Exercices 20

1.8 Dégénérescences 20

1.8.1 Dégénérescence de première espèce 20

1.8.2 Dégénérescence de seconde espèce 21

2 Méthodes d'optimisation combinatoire en programmation mathématique **23**

2.1 Introduction 23

2.2 Programmation linéaire en variables 24

2.3 Programmation linéaire mixte 24

ISTA

2.4 Programmation quadratique en variables 25
2.5 Linéarisation en variables 25
2.6 Dualité 26
2.7 Relaxation Lagrangienne 29
2.7.1 Résolution du problème dual Lagrangien 30
2.8 Exemple 32
2.8.1 Relaxation Lagrangienne vs. relaxation linéaire 35
2.8.2 Applications de la relaxation Lagrangienne 36
2.9 Méthodes de résolution 37
2.9.1 Méthodes exactes 37
2.10 Méthodes approchées 41

3 Notions de base en complexité des algorithmes 44
3.1 Rappel d'algorithmique 44
3.2 Paradigmes d'algorithmique 49
3.3 Exemples 50
3.4 Complexité des algorithmes / des problèmes 51
3.5 Calcul de la complexité 53
3.6 Notation de Landau 54
3.7 Complexités usuelles 54
3.8 Classes de problème 54
3.9 Classification des problèmes de décision:P, NP, NP-complet, NP-difficile. 56

ISTA

Objectif General

Savoir un problème d'optimisation et comment concevoir et implémenter des heuristiques pour résoudre des problèmes difficiles.

Objectifs Spécifiques

- connaître la différence entre "heuristique" et "méta-heuristique",
- comprendre la classification générale des méthodes d'optimisation combinatoire et les concepts sous-jacents,
- décrire le fonctionnement des méthodes classiques
- modéliser un problème et lui appliquer une méthode d'optimisation avec un peu plus de temps et de pratique,
- évaluer et comparer plusieurs méthodes d'optimisation sur un problème donné,
- combiner différentes méthodes de manière performante.

ISTA

Chapter 1

Quelques notions de rappel

1.1 Programmation linéaire

Proposée par George Bernard Dantzig en 1949, la programmation linéaire a permis la résolution des problèmes d'optimisation dans la plupart des domaines.

La programmation linéaire « PL » est l'une des techniques de la recherche opérationnelle la plus répandue et la plus utilisée. Comme tout problème de programmation, un problème de PL recherche une utilisation efficace de ressources rares qui permettra de satisfaire des objectifs fixés. Les problèmes de PL sont caractérisés par un grand nombre de solutions pouvant satisfaire les conditions de base du problème.

En effet, une fois un problème modélisé sous la forme d'équations linéaires, des méthodes assurent la résolution du problème de manière exacte.

Le choix d'une solution particulière comme solution optimale (meilleur solution) dépend du but ou de l'objectif global indiqué dans l'énoncé du problème. Une solution optimale est une solution qui satisfait les conditions du problème et de l'objectif fixé.

ISIA

1.2 Quelques rappels

1.2.1 Ecriture matricielle d'un système d'équations du premier degré à plusieurs inconnues

Un système de n équations linéaires à p inconnues $(n, p \in N^*)$ est un système de la forme:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1j}x_j + \cdots + a_{1p}x_p = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2j}x_j + \cdots + a_{2p}x_p = b_2 \\ \vdots a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nj}x_j + \cdots + a_{np}x_p = b_n \end{cases} \tag{1.1}$$

dans lequel $\forall i \in (1, \cdots, n), \forall j \in (1, \cdots, p)$, a_{ij} et b_i sont des éléments du corps $\mathbb{R}$ et les x_j sont des inconnues à déterminer dans $\mathbb{R}$.

Une solution du système (1.1) est une suite $(x_1, x_2, \cdots, x_p)$ vérifiant toutes les équation de ce système. Si $\forall i \in (1, \cdots, n), b_i = 0$, le système (1.1) est dit, système homogène. Lorsqu'au moins un terme indépendant est non nul, le système est dit non homogène. Si on pose pour le système (1.1)

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2p} \\ \vdots & & & & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nj} & \cdots & a_{np} \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_j \\ \vdots \\ b_n \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_j \\ \vdots \\ x_p \end{pmatrix}$$

Le système (1.1) devient :

$$\underbrace{\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2p} \\ \vdots & & & & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nj} & \cdots & a_{np} \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_j \\ \vdots \\ x_p \end{pmatrix}}_{x} = \underbrace{\begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_j \\ \vdots \\ b_n \end{pmatrix}}_{b}$$

ce qui équivaut à $Ax = b$ (forme matricielle).

Example 1.1 *L'écriture sous forme matricielle du système*

$$\begin{cases} 2x_1 + 3x_2 - x_3 = 4 \\ 4x_1 - x_2 = 5 \\ x_1 + 5x_2 + 2x_3 = 1 \end{cases} \tag{1.2}$$

est

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & -1 & \\ 1 & 5 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 5 \\ 1 \end{pmatrix}. \tag{1.3}$$

Si l'on suppose que la matrice A est régulière c'est-à-dire que son inverse existe. De la relation de la forme matricielle, la solution du système matriciel est donnée par

$$x = A^{-1}b.$$

Example 1.2 *Résoudre le système suivant par la méthode d'inversion d'une matrice :*

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 5 \\ 2x_1 - x_2 + 4x_3 = 11 \\ -x_2 + x_3 = 3 \end{cases} \tag{1.4}$$

Sa forme matricielle et sa solution sont données par

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & -1 & 4 \\ 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \\ 3 \end{pmatrix}$$

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & -1 & 4 \\ 0 & -1 & 1 \end{pmatrix}^{-1} \begin{pmatrix} 5 \\ 11 \\ 3 \end{pmatrix}$$

$$\Rightarrow \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \left\{ \begin{pmatrix} 1 \\ -1 \\ 2 \end{pmatrix} \right\}$$

1.2.2 Rang d'une matrice

Une matrice A du type $A_{m\times n}$ est de rang r ssi une matrice d'ordre r extraite de A est régulière et toutes les matrices d'ordre $(r+1)$ extraites de A sont singulières. Autrement dit, le « rang » d'une matrice A , noté $rg(A)$, est le nombre de lignes ou de colonnes n , de la plus grande sous-matrice carrée $n \times n$ de la matrice A de déterminant non nul.

La plus grande sous-matrice carrée d ' une matrice de taille $n \times n$ est de taille le minimum entre m et n. Par exemple, dans le cas de la matrice de taille 4×2 suivante la plus grande sous-matrice carrée est de taille 2×2

$$\begin{pmatrix} 3 & 7 \\ 0 & 8 \\ 1 & -1 \\ 9 & 7 \end{pmatrix} \tag{1.5}$$

Et de même, pour une matrice de taille 3×5, la plus grande sous-matrice carrée est de taille 3×3 .

$$\begin{pmatrix} 3 & 7 & 2 & 6 & 1 \\ -2 & 8 & 9 & 0 & 1 \\ 13 & 5 & 12 & 6 & 10 \end{pmatrix} \tag{1.6}$$

Ainsi, la taille maximale d ' une sous-matrice carrée est donnée par le minimum du nombre de ligne et de colonnes. La plus petite valeur possible pour le rang est 0, mais elle ne peut être atteinte que dans le cas où il est impossible de trouver une matrice 1×1 de déterminant non nul, c'est-à-dire dans le cas où la matrice a tous ses coefficients nuls.

Exercices Déterminer le rang des matrices suivantes

1.

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} 5 & 5 & 1 \\ 6 & 6 & 1 \\ 5 & 5 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 2 & 4 \\ -2 & -4 & -8 \\ 3 & 6 & 12 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 1 & \alpha \\ 2 & 0 \end{pmatrix}, \alpha = 0, \alpha \neq 0.$$

,

Exemples

1. $A = \begin{pmatrix} 1 & 2 & 0 \end{pmatrix}$
   Les D_p sont $|1|$, $|2|$, donc $rg(A) = 1$.

2. $A = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \end{pmatrix}$, $D_p = \begin{vmatrix} 1 & 0 \\ 2 & 3 \end{vmatrix} = 3 \neq 0$ donc $rg(A) = 2$.

3. $A = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 5 & 6 & 8 \end{pmatrix}$, $D_p = \begin{vmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 5 & 6 & 8 \end{vmatrix} = |A| \neq 0$ donc $rg(A) = 3$

Théorème 1.1 *Le rang d'une matrice ne change pas si:*

- *on permute deux rangées parallèles (deux colonnes ou deux lignes)*
- *on remplace l'une des ses colonnes par une combinaison linéaire des autres colonnes (de même pour les lignes).*

Exemples

1. Déterminer le rang de

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 2 & -1 \end{pmatrix} \tag{1.7}$$

$$\begin{aligned} rg(A) &= rg \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 2 & -1 \end{pmatrix} \\ &= rg \begin{pmatrix} 1 & 0 & 1 & -1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 2 & -1 \end{pmatrix} \\ &= rg \begin{pmatrix} 1 & 0 & 1 & -1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix} \\ &= rg \begin{pmatrix} 1 & 0 & 1 & -1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{aligned}$$

   Donc $rang(A) = 2$.

2.

$$B = \begin{pmatrix} 1 & 2 & -1 & -4 \\ 1 & 3 & 1 & 7 \\ 2 & 3 & -4 & 5 \\ 2 & 3 & -8 & -1 \end{pmatrix} \tag{1.8}$$

Théorème 1.2 *On a* $rg(A) = 0$ *si et seulement si* A *est la matrice zéro* o_{mn} .

1.2.3 Base d'une matrice

Soit A une matrice $m \times n, x \in \mathcal{R}^n, b \in \mathcal{R}^n$. On veut résoudre le système $Ax = b$. On pose $M = (A|b)$.

Definition 1.1 *Une base de A est toute sous-matrice régulière B de A. Il faut que la matrice A (de type $m \times n$) soit de rang m. Autrement dit, une base de A est toute sous matrice B formée de colonnes linéairement indépendantes de la matrice A.*

Example 1.3

$$\begin{pmatrix} 2 & 0 & 4 & 1 & 0 \\ 4 & 1 & 7 & 1 & 0 \end{pmatrix} \tag{1.9}$$

$$B = \begin{pmatrix} 2 & 0 \\ 4 & 1 \end{pmatrix}, B_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \cdots, B_2 = \begin{pmatrix} 0 & 0 \\ 1 & 2 \end{pmatrix}, \cdots n'est\ pas\ une\ base. \tag{1.10}$$

Prenons la sous-matrice B et soit H l'ensemble des autres colonnes de A: $A = (B|H)$ et $\begin{pmatrix} x_B \\ x_H \end{pmatrix}$ où les x_B sont les variables de base et les x_H sont des variables hors base.

Le système $Ax = b$ devient

$$\begin{aligned} Ax = b &\iff (B|H)\begin{pmatrix} x_B \\ x_H \end{pmatrix} = b \\ &\iff Bx_B + Hx_H = b \\ &\iff x_B + B^{-1}Hx_H = B^{-1}b \end{aligned} \tag{1.11}$$

Si on pose $x_H = 0$, on obtient la solution particulière $\bar{x}_B = B^{-1}b$.

On dit que $\begin{pmatrix} B^{-1}b \\ 0 \end{pmatrix} = \begin{pmatrix} \bar{x}_B \\ x_H \end{pmatrix}$ est la solution de base associée à la base B. Lorsque $\bar{x}_B \geq 0$, on a une solution de base admissible ou réalisable. Si une des composantes de x_B est négative, on dit que la solution de base dégénérée.

Exemple

Soit le système

$$\begin{cases} 2x_1 + x_2 + x_3 = 6 \\ x_1 + 2x_2 + x_4 = 6. \end{cases} \tag{1.12}$$

Sous forme matricielle, on a

$$\begin{pmatrix} 2 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \end{pmatrix}, \tag{1.13}$$

ISTA

ce qui implique que

$$A = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 1 & 2 & 0 & 1 \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}, b = \begin{pmatrix} 6 \\ 6 \end{pmatrix}. \tag{1.14}$$

- Pour la base $B_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$,on a $\bar{x}_{B_1} = B_1^{-1}b = \begin{pmatrix} 6 \\ 6 \end{pmatrix}, \bar{x}_H = 0$. Ce qui implique que $x = \begin{pmatrix} \bar{x}_{B_1} \\ \bar{x}_H \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \\ 0 \\ 0 \end{pmatrix}$, qui est la solution de base associée à la base $B_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

1.3 Rappel de géométrie

Soit A un vecteur de $\mathcal{R}^n$ ou une matrice.

Definition 1.2 *Un hyperplan noté*

$$H(A, a) = \{x \in \mathbb{R}^n / Ax = a\}$$

sépare l'espace en deux demi-espaces:$H^+ = \{x \in \mathbb{R}^n / Ax \geq a\}$ *et* $H^- = \{x \in \mathbb{R}^n / Ax < a\}$.

Exemple

Hyperplan en deux dimensions : soit un vecteur $A = (2, 1)$ *et un entier* $a = 2$. *On a* $Ax = a \Longleftrightarrow 2x + y = 2$ *qui est une équation d'une droite.*

Definition 1.3 *Combinaison convexe* P. *Soit* $u_1 \cdots u_m \in \mathbb{R}$ *et* $\alpha_1, \cdots, \alpha_m \in \mathbb{R}$ *avec* $\sum_{k=1}^{m} \alpha_k = 1$. *Alors* $\alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_m u_m$. *est une combinaison convexe de* $u_1, \cdots, u_m$

Une partie C *de* $\mathbb{R}^n$ *est convexe si pour toute paire de points* x *et* y *de* C, *toute combinaison convexe de* x *et* y *est dans* C *; ce qui revient à dire que le segment qui joint* x *et* y *est entièrement contenu dans* C.

Autrement dit une partie C *de* $\mathbb{R}^n$ *est dite convexe si et seulement si* $\forall x, y \in C, \forall \alpha \in [0, 1] : \alpha x + (1 - \alpha)y \in C$.

ISTA

Géométriquement, $p(\alpha) = \alpha x + (1-\alpha)y$ *est le barycentre des points* x *et* y *affectés des coefficients (ou poids) respectifs* α *et* 1α*. Lorsque* α *décrit l'intervalle* $[0,1]$*,* $p(\alpha)$ *décrit le segment fermé d'extrémités* x *et* y *que l'on notera* $[x,y]$*. Dire que* $4C$ *est convexe, c'est donc dire que* C *contient un segment dès qu'elle contient ses extrémités. En fait un convexe contient toute combinaison convexe de ses points.*

Definition 1.4 *Un point* x *d'un ensemble convexe est un point extrême s'il ne peut être exprimé comme combinaison linéaire convexe de deux points de* C*.*

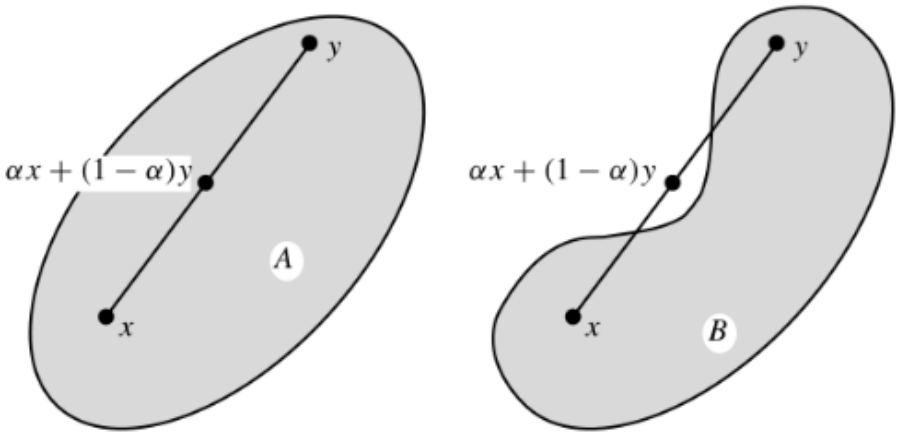


Figure 1.1: Ensemble convexe : A est convexe, B n'est pas convexe

Definition 1.5 *Soit* S *une partie convexe de* $\mathbb{R}^n$ *. L'enveloppe convexe de* S *notée* $C(S)$ *ou* $conv(S)$ *est l'ensemble de toutes les combinaisons linéaires convexes des points extrêmes de* S*.* S *comporte un nombre fini de points extrêmes, cette enveloppe est appelée polyèdre convexe ou simplexe.*

Proposition 1.1 *L'enveloppe convexe de* S *est l'ensemble de toutes les combinaisons convexes finies d'éléments de* S *:*

$$conv(S) = \{\sum_{i=1}^{n} \alpha_i u_i / n \geq 1, u_i \in S, \alpha_i \geq 0, \sum_{i=1}^{n} \alpha_i = 1\}$$

1.4 Formulation générale d'un problème de programmation linéaire "PPL"

Dans un problème de PL, on recherche l'optimum (minimum, maximum) d'une fonction linéaire des variables devant satisfaire des relations (équations ou inégalités) appelées contraintes.

La forme générale d'un PPL est

$$Optimiser(maximiser\ ou\ minimiser) z = Cx \tag{1.15}$$

$$s/c$$

$$Ax(\leq, =\geq)b \tag{1.16}$$

$$x \geq 0 \tag{1.17}$$

avec $x = n \times 1$matrice colonne, $A = m \times n$ matrice,$b = m \times 1$ matrice colonne,$c = 1 \times m$ matrice ligne.

Donc

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \vdots & \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad C = \begin{pmatrix} c_1 & c_2 & \cdots & c_n \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix},$$

ave a_{ij}, b_i, c_j sont des constantes réelles connues. L'expression (1.15) est appelée fonction objectif ou fonction économique. Les inégalités (1.16) sont appelées les contraintes et la relation (1.17) est appelée « contraintes de non-négativité » du PPL.

La fonction objectif est à maximiser ou à minimiser. Minimiser une fonction $f(x)$ revient à maximiser $-f(x)$; c'est à dire

$$minf(x) = max(-f(x)).$$

Les relations (1.15), (1.16), (1.17) peuvent être écrites explicitement :

$$Optimiser(maximiser\ ou\ minimiser)\ z = C_1x_1 + \cdots + C_1x_n \tag{1.18}$$

$$s/c$$

$$a_{11}x_1 + \cdots + a_{1n}x_n(\leq, =, \geq)b_1 \tag{1.19}$$

$$\vdots$$

$$a_{m1}x_1 + \cdots + a_{mn}x_n(\leq, =, \geq)b_n \tag{1.20}$$

$$x_1 + x_2 + \cdots + x_n \geq 0 \tag{1.21}$$

- z représente la fonction objectif ou économique,
- $x_1, x_2, \cdots, x_n$ représentent les variables de décision,
- $c_1, c_2, \cdots, c_n$ représentent les coefficients des variables de décision appelés aussi « coûts »,
- $a_{i1}, a_{i2}, \cdots, a_{in}$ sont appelés des coefficients technologiques.

ISTA

On définit les contraintes comme étant des relations exprimant la disponibilité d'unités de ressources. Dans la plus part des problèmes, on ne peut pas conduire une activité à un niveau négatif ; d'où la présence des contraintes de non-négativité.

La modélisation en PL, suit les étapes suivantes:

Etape 1. Identification des variables de décision

Etape 2. Formulation des contraintes et identification des ressources associées,

Etape 3. Vérification de la linéarité des contraintes,

Etape 4. Formalisation du modèle. Résoudre un PL c'est déterminer les valeurs des variables x_j soumises aux contraintes du programme en rendant optimale la fonction économique.

1.5 Exemples de programme linéaire

1.5.1 Le problème de la nourriture pour chiens

Soit une entreprise de nourriture pour chiens, qui fabrique 2 types de granulés: le Wag-Tail (W) et le Bark-Mad (B). Chacun des types utilise un mélange de légumes, boeuf et poisson, dans les proportions suivantes.

ISTA

Ingrédient	Qté totale	Qté dans B	Qté dans W
Légumes	1400 kg	4 kg	4 kg
Poisson	1800 kg	6 kg	3 kg
Bœuf	1800 kg	2 kg	6 kg

On suppose que la compagnie opère un bénéfice de 12 euros sur chaque paquet de B et de 8 euros sur ceux de W. Comment l'entreprise peut-elle faire pour maximiser son profit ?

Modélisation du problème

On note qu'on cherche à maximiser une quantité en fonction de diverses contraintes. Il s'agit bien d'un problème d'optimisation. On peut exprimer le problème de la manière suivante:

Etape 1. Variables de décision

- x_1 le nombre de paquet de Bark-Mad produits,
- x_2 le nombre de paquet de Wag-Tail produits.

Etape 2. Formulation des contraintes

- $4x_1 + 4x_2 \leq 1400$: Légumes
- $6x_1 + 3x_2 \leq 1800$: Poisson
- $2x_1 + 6x_2 \leq 1800$: Viande
- Finalement $x_1 \geq 0$ et $x_2 \geq 0$. Du point de vue de l'optimisation du profit, on cherche à maximiser
$$z = 12x_1 + 8x_2.$$

Etape 3. Vérification de la linéarité des contraintes ainsi que la fonction objectif
Toutes les contraintes ainsi que la fonction objectif sont linéaires.

Etape 4. Formulation mathématique

$$\begin{cases} max\ z = 12x_1 + 8x_2 \\ s/c \\ 4x_1 + 4x_2 \leq 1400 \\ 6x_1 + 3x_2 \leq 1800 \\ 2x_1 + 6x_2 \leq 1800 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.22}$$

ISTA

On constate que, sous cette forme, toutes les contraintes ainsi que la fonction de profit s'expriment de manière linéaires. Lorsqu'on cherche à maximiser une telle fonction objectif qui s'exprime de manière linéaire en fonction des variables du problèmes, sous certaines contraintes également exprimées de manière linéaire, on parle de programmation linéaire.

1.5.2 Gestion d'une brasserie

Un brasseur de bière dispose de 240kg de maïs, de 5kg de houblon et de 595kg de malt. Il peut produire aussi bien de la bière blonde que de la bière brune. Un tonneau de bière blonde nécessite 2, 5kg de maïs, 125g de houblon et 17, 5kg de malt. Chaque tonneau produit un bénéfice net de 65 euros. Un tonneau de bière brune nécessite 7, 5kg de maïs, 125g de houblon et 10kg de malt. Chaque tonneau produit un bénéfice net de 115 euros. On suppose que les autres ingrédients (comme l'eau) sont disponibles en quantité illimitée: on ne tiendra donc pas compte de leurs coûts.

Quel est, pour le brasseur, le meilleur plan de production ? En d'autres termes, quel est le plan de production lui fournissant le bénéfice maximum avec les stocks disponibles ?

Modélisation du problème

Etape 1. Variables de décision

- x_1 le nombre de tonneau de bière blonde,
- x_2 le nombre de tonneaux de bière brune. Evidemment $x_1 \geq 0, x_2 \geq 0$ et ces variables sont entières, mais nous allons les supposer réelles.

Etape 2. Formulation des contraintes,

- $2,5x_1 + 7,5x_2 \leq 240$: la quantité de maïs disponible,
- $0,125x_1 + 0,125x_2 \leq 5$: la quantité de houblon disponible,
- $17,5x_1 + 10x_2 \leq 595$: la quantité de malt disponible,
- Finalement $x_1 \geq 0$ et $x_2 \geq 0$. L'objectif du brasseur est de maximiser la fonction :

$$z(x_1, x_2) = 65x_1 + 115x_2$$

Etape 3. Vérification de la linéarité des contraintes ainsi que la fonction objectif.

Toutes les contraintes ainsi que la fonction objectif sont linéaires.

Etape 4. Formulation mathématique du problème.

On est donc ramené à résoudre le problème

ISTA

$$\begin{cases} max\ z(x_1, x_2) = 65x_1 + 115x_2 \\ s/c \\ 2,5x_1 + 7,5x_2 \leq 240 \\ 0,125x_1 + 0,125x_2 \leq 5 \\ 17,5x_1 + 10x_2 \leq 595 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.23}$$

ou simplement

$$\begin{cases} max\ z(x_1, x_2) = 65x_1 + 115x_2 \\ s/c \\ x_1 + 3x_2 \leq 96 \\ x_1 + x_2 \leq 40 \\ 3,5x_1 + 2x_2 \leq 119 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.24}$$

1.6 Exercices

1. Une usine fabrique 2 produits A et B en utilisant deux matières premières α et β. Le produit A coûte 4F et le produit B coûte 5F . On dispose d'un stock de 8kg de matièresα et de 7kg de matières β. Pour produire une unité du produit A, on a besoin de 2kg de matières α et 1kg de matière β ; tandis qu'une unité du produit B est obtenue à partir de 1kg de matières α et de 2kg de matières β. Par la modélisation,donner un programme de production pour l'usine qui maximise son profit sachant qu'on ne peut pas produire plus de 3 unités du produit B.

2. Un boulanger fabrique du pain, des croissants, des gâteaux et des beignets. Chaque produit exige un certain nombre d'heures pour la phase de préparation et pour la phase de cuisson. Le temps disponible à chaque phase est limité à un certain nombre d'heures. Le boulanger a l'information suivante concernant la réalisation de chaque produit ainsi que le profit par unité vendue: Le temps disponible à la phase de pré-

	Pain	croissants	gâteaux	beignets
Bénéfice	6	15	40	25
Heures pr. phase de préparation	1,5	3	8	6
Heures pr. phase de cuisson	9,5	1	12	9

paration est de de 800h/mois et 600h/mois pour la phase de cuisson . On demande :

a. de formuler le programme linéaire qui maximiserait les bénéfices tout en excédant pas les heures disponibles,

b. d'écrire le programme linéaire sous forme matricielle.

3. Un bijoutier désire fabriquer 3 types de bijoux A, B et C. Les prix unitaires de ces bijoux sont respectivement 350, 200 et 400. La composition minimale en or est de 900, en argent de 1000 et en cuivre de 1200 dans les unités appropriées. Le tableau suivant indique la composition minimale de chaque bijou:

	or	argent	cuivre
A	1	1	3
B	2	3	2
C	3	2	1

a. Formuler le programme linéaire permettant de déterminer les quantités des 3 bijoux à fabriquer de façon à maximiser le coût total tout en respectant les compositions des données.

b. Donner le résultat de a) en écriture matricielle.

4. Une usine de pièces doit fabriquer 4 types de pièces P_1, P_2, P_3 et $P4$. Elle s'approvisionne à partir de 2 entrepôts E_1 et E_2 . Le tableau suivant indique les quantités à prendre dans chaque entrepôt pour chaque pièce, les prix unitaires et les minima exigés pour chaque pièce: Formuler le programme linéaire permettant de déterminer les quantités à prendre dans les entités E_1 et E_2 de façon à optimiser le coût total et donner le résultat en écriture matricielle.

	P_1	P_2	P_3	P_4	Prix unitaire
E_1	2	3	1	2	200
E_2	1	2	3	2	300
Minima	7	12	9	15	

ISTA

1.7 Méthodes de résolution en programmation linéaire

1.7.1 Méthode graphique

La méthode graphique peut être appliquée à un PPL contenant uniquement deux variables inconnues.

Nous savons qu'une droite linéaire à 2 variables représente une droite dans $\mathbb{R}^2$, laquelle droite divise le plan en deux demi-plans. Les contraintes définissent des demi-plans de $\mathbb{R}^2$ (on trace les droites) dont les équations

sont tirées des contraintes en remplaçant l'inégalité par l'égalité ; et on élimine la partie non désirée. L'intersection de ces demi-plans constitue la région admissible. Un point $p(x_1, x_2)$ est appelée solution admissible (solution réalisable) si p satisfait toutes les contraintes (appartient à la région admissible).

La fonction objectif est une droite dans $\mathbb{R}^2$ de direction $\vec{c} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$.Tout déplacement dans le sens du vecteur maximise la fonction économique tandis que tout déplacement dans le sens inverse la minimise. Supposons que $\mathcal{D}$ est une intersection d'un nombre fini de demi-plans, alors trois cas peuvent se présenter :

- $\mathcal{D}$ peut être vide,
- $\mathcal{D}$ peut être un ensemble convexe borné et fermé possédant un nombre fini de points extrêmes, càd un polyèdre convexe,
- $\mathcal{D}$ peut être un ensemble convexe non-borné.

Si $\mathcal{D}$ est vide alors le PPL est non réalisable. Ainsi la question de solution optimale n'apparaît pas. Dans le second cas, lorsque $\mathcal{D}$ est un polyèdre convexe, nous avons une solution optimale finie grâce au théorème suivant:

Théorème 1.3 *Tout maximum ou minimum de la fonction objectif d'un PPL est atteint à un des points extrêmes de l'ensemble $\mathcal{D}$.*

Dans le troisième cas, nous pouvons avoir aussi une solution optimale finie ou une solution non-bornée et dans ce cas, la valeur tends vers l'infini.

En résumé, deux situations peuvent se présenter :

- la solution optimale est un point p appartenant à l'intersection de 2 côtés du polygone convexe représentant la région admissible : on l'appelle point extrême. Dans ce cas, la solution optimale est unique,
- la solution optimale est un côté du polygone dont les côtés sont les segments de droite représentant les contraintes linéaires du problème. Dans ce cas, on a une infinité de solutions optimales (mais la valeur de la fonction objectif est unique).

  Donc, de ce qui précède, Si la région admissible est bornée :

- la solution optimale est un point extrême : on a une solution unique,
- la solution optimale est un côté du polygone : on a une infinité de solutions (valeur de la fonction optimale unique) ;

Théorème 1.4 *Si l'ensemble des contraintes d'un programme linéaire forme un polyèdre 1 non vide, il existe (au moins) une solution optimale qui est un sommet de ce polyèdre.*

∗ Si la région admissible est non bornée :

- la solution optimale est un point extrême : on a une solution unique,
- la solution est un côté de la région admissible: on a une infinité de solution (valeur de la fonction objective est unique),
- pas de solution optimale finie ;

∗ si la région admissible est vide :

- l'intersection des demi-plans est vide
- les contraintes de non-négativité ne sont pas satisfaites.

Example 1.4

$$\begin{cases} maxz(x_1, x_2) = 4x_1 + 12x_2 \\ s/c \\ x_1 \leq 1000 \\ x_2 \leq 5000 \\ \frac{1}{500}x_1 + \frac{1}{25}x_2 \leq 35 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.25}$$

ou simplement

$$\begin{cases} max\ z(x_1, x_2) = 4x_1 + 12x_2 \\ s/c \\ x_1 \leq 1000 \\ x_2 \leq 5000 \\ x_1 + 2x_2 \leq 1750 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.26}$$

Rien ne nous empêche ici de recourir à la méthode géométrique ou graphique, puisque nous n'avons que deux variables et qu'il est facile de représenter le problème dans l'espace à deux dimensions (le plan) dont les axes porteront les quantités x_1 et x_2 . Toute solution du problème est représentée par un point du plan, de coordonnées x_1 et x_2 , déterminées. Les contraintes $x_1 \geq 0$ et $x_2 \geq 0$ font que nous pourrons nous contenter du premier quadrant du plan dans lequel les deux coordonnées sont positives ou nulles.

Les contraintes sont aisées à représenter. Ainsi la contrainte $x_1 \leq 1000$ signifie que dans le quadrant positifs les points dont la coordonnée x_1 excède 1000 sont exclus du domaine des solutions. Or, $x_1 \leq 1000$ représente une droite D_1 verticale, orthogonale à l'axe Ox_1 au point $x_1 \leq 1000$: à toutes les solutions respectant cette contrainte, sont associés des points compris entre D_1 et l'axe Ox_2 (et au-dessus de l'axe Ox_1). On raisonnerait de même pour la contrainte $x_2 \leq 500$ (la droite D_2 , horizontale, a pour équation $x_2 \leq 500$.

ISTA

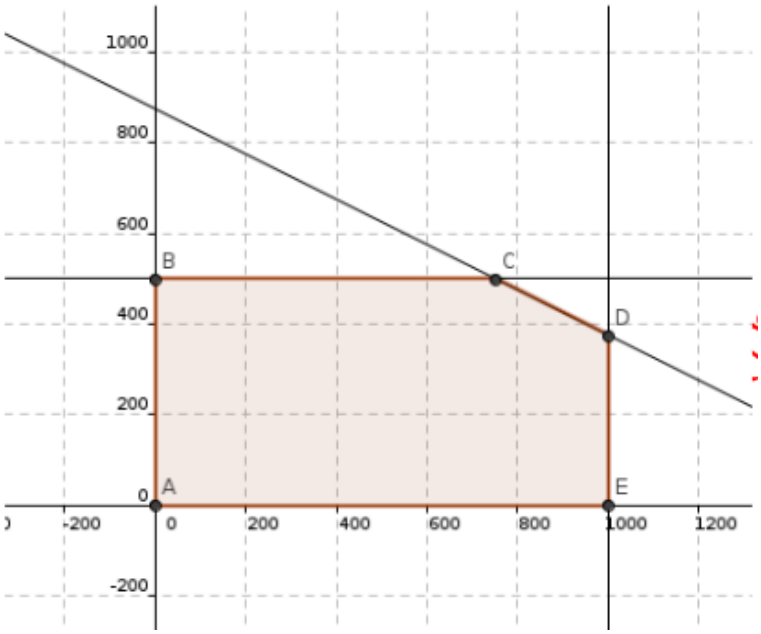
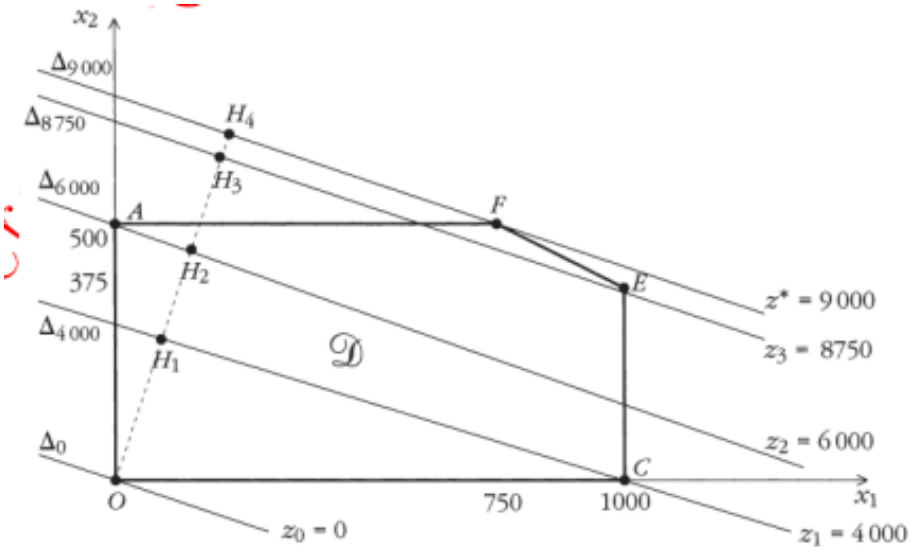


Figure 1.2: Représentation graphique du modèle mathématique (1.26)

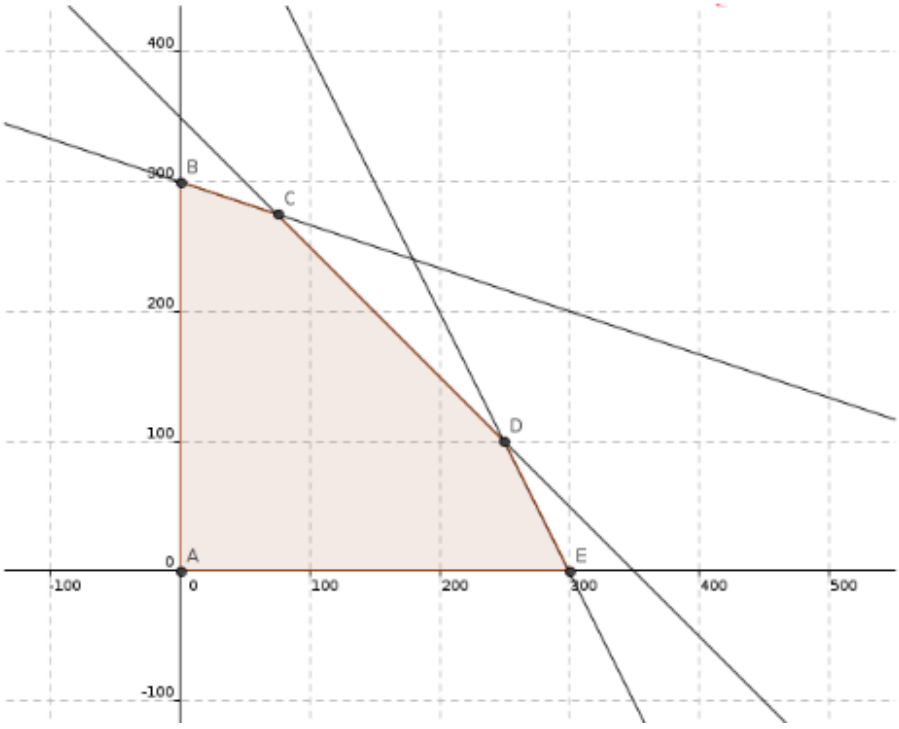
Si le problème ne comportait que les contraintes $0 \leq x_1 \leq 1000$ et $0 \leq x_2 \leq 500$ le domaine « admissible », c'est-à-dire le domaine des points associés aux solutions vérifiant ces contraintes, serait le rectangle $OABC$. Mais $x_1 + 2x_2 \leq 1750$ est aussi l'équation d'une droite D_3 dont l'intersection avec la droite D_1 est le point $E(x_1 = 1000, x_2 = 375)$ et l'intersection avec D_2 est le point $F(x_1 = 750, x_2 = 500)$. Le domaine des solutions admissibles D de notre problème est donc le polygone convexe $OAFEC$, à la frontière ou à l'intérieur duquel se trouvent les points associés à ces solutions admissibles. Il est facile de voir que la droite qui passe par F est obtenue pour $\alpha = 9000$; cette droite a un seul point en commun avec le polygone: c'est le point F. D'où la solution optimale: $x_1^* = 750, x_2^* = 500, z^* = 9000$. Cette méthode géométrique permet de résoudre tous les programmes linéaires à deux variables ; en résumé, elle consiste à écarter la droite de la fonction économique passant par un point initial évident (ici O), parallèlement à elle-même, tant que l'intersection de la droite courante avec le polygone D des solutions admissibles n'est pas vide. La résolution s'arrête lorsque la droite courante n'a plus qu'un seul point de contact (ou une arête de contact) avec ce polygone (on a évidemment supposé que D n'est pas vide c-à-d que les contraintes ne sont pas contradictoires).

NB: Si on avait $z = 4x_1 + 8x_2$, les droites Δ_α seraient parallèles à FE : alors tous les points de l'arête seraient optimaux, y compris les sommets F et E.



Example 1.5 *Donner la solution du PPL suivant par la méthode graphique*

$$\begin{cases} max\ P = 12x_1 + 8x_2 \\ s/c \\ 4x_1 + 4x_2 \leq 1400 \\ 6x_1 + 3x_2 \leq 1800 \\ 2x_1 + 6x_2 \leq 1800 \\ x_1 \geq 0, x_2 \geq 0. \end{cases}$$



On a d'autre part représenté la fonction de profit (ou fonction objectif), que l'on cherche à maximiser, par un ensemble de droites d'équation $12x_1+8x_2 = z$, avec z variable. Toutes ces droites sont donc parallèles entre elles. Sur le graphique, on a représenté la droite avec $z = 0$ associée avec un vecteur perpendiculaire à la droite indiquant dans quel sens z grandit.

Optimiser ce système revient donc à trouver P maximal, c'est-à-dire un couple (x_1, x_2) tel que z soit le plus grand possible. Un raisonnement géométrique élémentaire nous incite à déplacer la droite $12x_1 + 8x_2 = z$ parallèlement à elle-même, dans le sens où z grandit, jusqu'à trouver l'intersection avec le simplexe telle que z soit le plus grand possible. Intuitivement, on s'attend, comme c'est le cas ici, à trouver l'optimum sur une frontière du simplexe. On trouve en effet le point M , de coordonnées $(x_1 = 250, x_2 = 100)$, réalisant un profit de $12 \times 250 + 8 \times 100 = 3800 euros$, ce qui est l'optimum.

1.7.2 Exercices

1.Donner la solution du PPL suivant par la méthode graphique

$$\begin{cases} max\ z(x_1, x_2) = 3x_1 + 2x_2 \\ s/c \\ x_1 + 2x_2 \leq 6 \\ 2x_1 + x_2 \leq 8 \\ -x_1 + x_2 \leq 1 \\ x_2 \leq 2 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.27}$$

2. Une entreprise fabrique 2 produits$X$ et Y . Pour sa conception, chaque produit fini nécessite 3 produits intermédiaires A, B et C. Pour fabriquer un produit X, on a besoin de 2 produits de A, de 2 produits de B et de 1 produit de C. De même, pour fabriquer un produit Y , on a besoin de 3 produits de A, de 1 produit de B et de 3 produits de C. En outre, l'entreprise dispose d'une quantité limitée de produits de A, B et C. Elle a 180 produits de A, 120 produits de B et 150 produits de C. Sachant que le prix de revient de X est de 3 francs et que celui de Y est de 4 francs, combien de produits X et Y faut-il fabriquer pour maximiser le profit ?

1.8 Dégénérescences

1.8.1 Dégénérescence de première espèce

Pour certains programmes linéaires, l'optimum peut être réalisé en plusieurs points de la frontière du domaine admissible: tous les points d'une arête ou

d'une facette (ou...) sont alors optimaux.

Definition 1.6 *On parle de dégénérescence de première espèce si le coefficient directeur de la droite représentant la fonction économique est identique à celui de la droite représentant une contrainte non redondante. Il existe donc une infinité de solutions.*

Ainsi soit le programme :

$$\begin{cases} maxz(x_1, x_2) = 6x_1 + 4x_2 \\ s/c \\ -3x_1 + 2x_2 \leq 4 \\ 3x_1 + 2x_2 \leq 16 \\ -x_1 + x_2 \leq 1 \\ x_1 \leq 3 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.28}$$

1.8.2 Dégénérescence de seconde espèce

Pour certains programmes linéaires, une (ou plusieurs) variable(s) de la base optimale peut être nulle.

Definition 1.7 *Une solution optimale est dite dégénérée si plus de deux contraintes concourent en ce point.*

Ce fait traduit que la fonction économique $z = 6x_1 + 4x_2$ est parallèle à l'hyperplan (ici, la droite) limitant le demi-plan déterminé par la contrainte $3x_1 + 2x_2 \leq 16$; les sommets B et C sont tous deux optimaux.
L'enveloppe convexe de ces sommets, c'est-à-dire le segment BC, est telle que chacun de ses points donne à z la valeur optimale 32. C'est le cas dit de « *dégénérescence de première espèce* ».

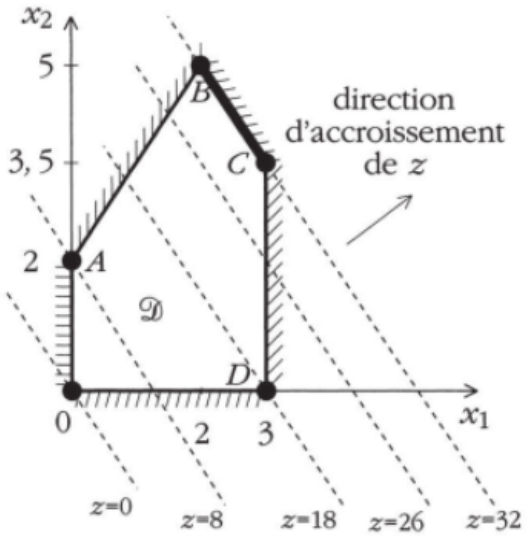


Figure 1.3: Dégénérescence de première espèce

Un exemple détaillé :

$$\begin{cases} max\ z(x_1, x_2) = x_1 + x_2 \\ s/c \\ x_1 + 4x_2 \leq 22 \\ -3x_1 + 2x_2 \leq 4 \\ 3x_1 + x_2 \leq 16 \\ x_1 \leq 3 \\ x_1 \geq 0, x_2 \geq 0. \end{cases} \tag{1.29}$$

Chaque fois qu'on obtient une ou plusieurs variables de base nulles, on est dans le cas de dégénérescence de seconde espèce. La raison géométrique en est qu'il passe, par un des sommets au moins, un ou plusieurs hyperplans (ici droites) supplémentaire (ici, le sommet B, au lieu de passer $n = 2$ droites, il en passe trois).

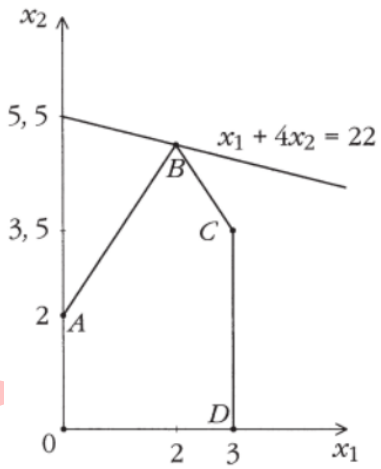


Figure 1.4: Dégénérescence de seconde espèce

Exercice Résoudre le problème de la programmation linéaire suivant en utilisant python

$$\begin{cases} maximize\ z = x + y \\ s/c \\ 6x + 4y \leq 24 \\ x + 2y \leq 6 \\ -x + y \leq 1 \\ y \leq 2 \\ x \leq 0 \\ y \leq 0 \end{cases}$$

Chapter 2

Méthodes d'optimisation combinatoire en programmation mathématique

2.1 Introduction

L'optimisation combinatoire (ou discrète) est une branche très importante en recherche opérationnelle, en mathématiques appliquées et en informatique. Elle comprend un grand nombre de problèmes d'optimisation combinatoire difficiles issus d'applications réelles dans différents domaines tels que l'industrie, la finance ou l'armée. L'objectif de ces problèmes combinatoires consiste à trouver une meilleure solution dans un espace fini et discret de solutions réalisables, qui respectent un ensemble de conditions, dites aussi contraintes. L'évaluation d'une solution est effectuée à l'aide d'une fonction dite fonction objectif. Une meilleure alternative (solution optimale) est une solution réalisable qui minimise ou maximise, selon le contexte, la fonction objectif.

Face à la résolution d'un problème d'optimisation, il est important d'identifier à quelle classe de problèmes il appartient. Nous nous intéressons particulièrement à la classe des problèmes d'optimisation discrète en variables 0-1. Les algorithmes conçus pour résoudre ce type de problèmes sont nombreux. Ils s'organisent en deux catégories : les algorithmes exactes qui permettent d'obtenir des solutions dont l'optimalité est garantie ; et les algorithmes approchés qui cherchent des solutions de bonne qualité, sans garantie d'optimalité, mais au profit d'un temps de calcul plus réduit.

Nous présentons les notions de base et les concepts utiles d'optimisation combinatoire. Nous illustrons également un ensemble de méthodes d'optimisation adaptées à la résolution des problèmes d'optimisation discrète en variables. Notons que nous allons considérer, pour les formules mathématiques, des notations matricielles et des notations générales (non matricielles)

INSA

2.2 Programmation linéaire en variables

Un problème de programmation linéaire (PL) en variables est un problème d'optimisation où les variables sont binaires et la fonction objectif et les contraintes sont toutes linéaires. Sa formulation mathématique, dans sa forme canonique, est la suivante:

$$\begin{cases} max \ \sum_{i=1}^{n} c_i x_i \\ s/c \\ \sum_{i=1}^{n} a_{ki} x_i \leq b_k, k = 1, \cdots, m \\ x_i \in \{0,1\}, i = 1, \cdots, n. \end{cases} \tag{2.1}$$

avec $c \in \mathbb{R}^n$ vecteur de coûts, $b \in \mathbb{Z}^m$ vecteur des termes constants des contraintes et $A \in \mathbb{Z}^{n \times m}$ matrice des coefficients des contraintes, où n est le nombre de variables et m est le nombre de contraintes.

Les éléments de l'ensemble $S = \{x \in \{0,1\}^n | Ax \leq b\}$ représentent les solutions réalisables du problème (un vecteur x qui respectent toutes les contraintes des équations 2 et 3 du système (2.1) . Les éléments de l'ensemble S qui maximisent la fonction objectif, donc l'équation 1 du systeme (2.1) sont les solutions optimales du modèle.

2.3 Programmation linéaire mixte

Un programme linéaire mixte en nombres entiers est un programme linéaire dans lequel les variables sont continues ou discrètes.

$$\begin{cases} max \ \sum_{i=1}^{n} c_i x_i + \sum_{j=1}^{p} d_j z_j \\ s/c \\ \sum_{i=1}^{n} a_{ki} x_i + \sum_{j=1}^{p} a'_{kj} x_j \leq b_k, k = 1, \cdots, m \\ x_i \in \{0,1\}, i = 1, \cdots, n, z_j \in \mathbb{R}_+, j = 1, \cdots, p, \end{cases} \tag{2.2}$$

avec $c \in \mathbb{R}^n, d \in \mathbb{R}^p, b \in \mathbb{Z}^m, A \in \mathbb{Z}^{n \times m}$ et $A' \in \mathbb{Z}^{p \times m}$, où n est le nombre de variables discrètes, p est le nombre de variables continues et m est le nombre de contraintes. Les deux vecteurs $x \in \{0,1\}$ n et $z \in \mathbb{R}_+^p$ représentent, respectivement, les variables discrètes et les variables continues.

$$S = \{0,1\}^n, z \in \mathbb{R}_+^p | Ax + A'z \leq b,$$

est l'ensemble des solutions réalisables.

ISTA

2.4 Programmation quadratique en variables

Un problème de programmation quadratique (PQ) est un problème d'optimisation dans lequel on minimise une fonction objectif quadratique, non nécessairement convexe, sur un polyèdre convexe. La forme mathématique la plus générale pour un problème quadratique sous contraintes linéaires en variables se présente de la manière suivante :

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} c_i x_i + \sum_{i=1}^{n} \sum_{\substack{j=1 \\ j \neq i}}^{n} q_{ij} x_i x_j \\ s.c \quad & \sum_{i=1}^{n} a_{ki} x_i \leq b_k \qquad k = 1, \ldots, m \\ & x_i \in \{0,1\} \qquad i = 1, \ldots, n \end{aligned}$$

où $Q_{n \times n}$ est une matrice carrée symétrique $(q_{ij} = q_{ji})$.

Rappelons qu'un domaine de solutions admissibles S d'un problème linéaire est dit convexe lorsque, pour tous points x_1 et x_2 de S, le segment $[x_1, x_2]$ est tout entier contenu dans S. Cela se traduit pour un domaine $S \in \mathbb{R}^n$ par la formule suivante :

$$\forall x_1, x_2 \in S, \forall \lambda \in [0,1], \lambda x_1 + (1-\lambda) x_2 \in S$$

2.5 Linéarisation en variables

Une panoplie de méthodes d'optimisation sont conçues pour résoudre des problèmes li- néaires (PL). Les appliquer sur un problème quadratique (PQ) nécessite une reformulation linéaire de la fonction objectif en introduisant des variables et des contraintes supplémen- taires. Dans la majorité des cas, une reformulation linéaire est effectuée pour résoudre des problèmes non linéaires par des méthodes exactes de programmation linéaire en variables mixtes. Le but est d'améliorer la borne inférieure en renforçant les relaxations utilisées dans un schéma classique de séparation et d'évaluation (Branch-and-Bound).

Nous nous intéressons à la résolution des problèmes quadratiques en variables par une heuristique de programmation linéaire en variables. Autrement dit, nous cherchons à reformuler la fonction objectif d'un problème quadratique en un problème linéaire en introduisant des variables.

La reformulation linéaire la plus utilisée est la linéarisation classique (ou standard). Elle consiste à remplacer chaque produit $x_i x_j$ par une variables

additionnelle y_{ij} et d'ajouter les contraintes suivantes:

$$y_{ij} \leq x_i, 1 \leq i < j \leq n \quad (2.3)$$
$$y_{ij} \leq x_j, 1 \leq i < j \leq n \quad (2.4)$$
$$y_{ij} \leq x_i + x_j - 1, 1 < i \leq j \leq n \quad (2.5)$$
$$y_{ij} \geq 0, 1 \leq i \leq j \leq n \quad (2.6)$$

Les contraintes (2.3)-(2.6) forcent la variable y_{ij} à la valeur zéro si et seulement si l'une au moins des deux variables x_i ou x_j est égale à zéro ($y_{ij} = 0 \Leftrightarrow x_i = 0 \vee x_j = 0$). De la même façon, elles forcent la variable y_{ij} à la valeur un si et seulement si les deux variables x_i et x_j sont égales à un ($y_{ij} = 1 \Leftrightarrow x_i = 1 \wedge x_j = 1$). La reformulation linéaire obtenue est alors :

$$\begin{cases} max \ \sum_{i=1}^{n} c_i x_i + \sum_{i=1}^{n} \sum_{j=1_{j \neq i}}^{n} q_{ij} y_{ij} \\ s/c \\ \sum_{i=1}^{n} a_{ki} x_i \leq b_k, k = 1, \cdots, m \\ x_i \in \{0,1\}, i = 1, \cdots, n, \\ z_j \in \mathbb{R}_+, j = 1, \cdots, p, \\ y_{ij} \leq x_i, 1 \leq i < j \leq n \\ y_{ij} \leq x_j, 1 \leq i < j \leq n \\ y_{ij} \leq x_i + x_j - 1, 1 < i \leq j \leq n \\ y_{ij} \geq 0, 1 \leq i \leq j \leq n \\ y_{ij} = y_{ji}, 1 \leq i \leq j \leq n \\ x_i, x_j \in \{0,1\}, i = 1, \cdots, n; j = 1, \cdots, n \end{cases} \quad (2.7)$$

Cela induit donc $\mathcal{O}(n^2)$ variables et $\mathcal{O}(n^2)$ contraintes. Signalons qu'il existe d'autres méthodes de linéarisation qui reformulent le problème quadratique en un problème linéaire en variables mixtes (continues et discrètes). Nous citons la méthode RLT (Reformulation Linearization Technique) qui ajoute $\mathcal{O}(n \times m)$ contraintes et la méthode compacte de Glover qui introduit $\mathcal{O}(n)$ contraintes.

ISTA

2.6 Dualité

A tout problème d'optimisation linéaire correspond un autre problème linéaire, impliquant les mêmes données. Le premier problème est appelé problème primal et le second est appelé problème dual. Le passage du primal au dual est effectué de manière à ce que chaque contrainte du primal devient une variable du dual et chaque variable du primal devient une contrainte du dual. Nous inversons les coefficients de la fonction objectif et des contraintes, et nous prenons la transposée de la matrice des contraintes. Ainsi,

Problème primal	problème dual
Objectif : maximisation	Objectif : minimisation
k^e contrainte d'inégalité ($\leq$)	k^e variable positive
k^e contrainte d'égalité	k^e variable quelconque
i^e variable positive	i^e contraintes d'inégalité ($\geq$)
i^e variable quelconque	i^e contrainte d'égalité
Coefficients de la fonction objectif : c_i	Termes constants des contraintes : b_i
Termes constants des contraintes : b_k	Coefficients de la fonction objectif : c_k
Matrice des coefficients de contraintes : A	Matrice des coefficients de contraintes : A^T

Figure 2.1: Transformation Primal-Dual avec les mêmes notations du problème (PL) à la section 2.4

les contraintes d'inéga- lité changent de sens. Le tableau 2.6 récapitule les différentes transformations du problème primal au problème dual.

Pour illustrer cette reformulation, nous associons au problème linéaire primal (PL) (cfr la section 2.2) le problème linéaire dual (2.1) suivant :

$$\begin{cases} min_\pi \ \sum_{k=1}^{n} b_k \pi_k \\ s/c \\ \sum_{i=1}^{n} a_{ki} \pi_k \geq c_i, ik = 1, \cdots, n \\ \pi_k \geq 0, i = 1, \cdots, m. \end{cases} \tag{2.8}$$

où sont les variables duales.

Selon les fondamentaux de la programmation linéaire, un problème linéaire est défini dans l'un (et seulement un) des trois cas : problème infaisable, problème non borné ou problème optimal. Pour un problème primal et son dual, seulement quelques combinaisons sont possibles. En effet, le théorème de dualité faible (théorème 2.1) montre que si le problème primal et son dual sont réalisables, alors ils sont bornés (chacun borne l'autre).

Théorème 2.1 *(Dualité faible) Pour un problème linéaire de maximisation, toute solution réalisable w_D du dual est une borne supérieure de n'importe quelle solution réalisable du problème primal w_P*

$$f_P(w_D) \geq f_D(w_P),$$

où f_P et f_D sont respectivement les fonctions objectifs du primal et du dual.

Une version plus forte du théorème de dualité (théorème 2.2) exclut la possibilité que l'un des problèmes soit infaisable et l'autre optimal.

Théorème 2.2 *Dualité forte) Si le problème primal admet une solution réalisable optimale* w_P *, alors le problème dual admet également une solution optimale* w_D *et nous avons*

$$f_P(w_D) = f_D(w_P).$$

Primal/Dual	optimal	non-borné	infaisable
optimal	possible (DF)	impossible (df)	impossible (DF)
non-borné	impossible (df)	impossible (df)	possible (df)
infaisable	impossible (DF)	possible (df)	possible

Figure 2.2: Lien primal-dual ((DF) signifie une dualité forte et (df) une dualité faible)

.

Toutes les autres combinaisons sont possibles. Le tableau 2.6 résume les différentes possibilités.

Exercices Donner le dual des problèmes suivants:

1.

$$\begin{aligned} Z : max\ & 16x_1 + 10x_2 + 4x_4 \\ & s/c \\ 8x_1 + 2x_2 + x_3 + 4x_4 &\leq 10 \\ x_1 + x_2 &\leq 1 \\ x_3 + x_4 &\leq 1 \\ x_i &\geq 0 \end{aligned}$$

2.

$$\begin{aligned} min\ G = 20x + 12y + 40z & \\ s/c & \\ x + y &\geq 20 \\ 2x + y + z &\geq 30 \\ x, y &\geq 0 \end{aligned}$$

3.

$$\begin{aligned} max\ z = 7x_1 + 12x_2 & \\ s/c & \\ 5x_1 - 2x_2 &\leq -11 \\ x_1 + 4x_2 &= 8 \\ x_1, x &\leq 0 \end{aligned}$$

ISTA

2.7 Relaxation Lagrangienne

La relaxation est une manipulation classique en optimisation combinatoire. Elle est appliquée pour fournir des bornes intéressantes (borne inférieure en minimisation et borne supérieure en maximisation) pour la résolution des problèmes linéaires en nombres entiers. Elle existe en deux types:

1. la relaxation des contraintes et en particulier la relaxation linéaire, permet d'élargir l'ensemble des solutions sur un espace plus large et facile à optimiser, en relâchant les contraintes d'intégrité.

$$x \in \{0,1\} \longrightarrow x \in [0,1].$$

2. Une autre piste consiste à modifier la fonction objectif. Une des techniques les plus utilisées est la Relaxation Lagrangienne. L'idée est de supprimer les contraintes difficiles en les intégrant dans la fonction objectif sous forme de combinaisons linéaires, où les coefficients sont des multiplicateurs de Lagrange (ou multiplicateurs Lagrangiens). Ces derniers pénalisent la fonction objectif si l'une des contraintes intégrées est violée.

Considérons le problème linéaire (P) en variables 0-1 suivant

$$Z \quad : \quad min \sum_{i=1}^{n} C_i x_i \tag{2.9}$$

$$s/c$$

$$\sum_{i=1}^{n} a_{ji} x_i \quad = \quad b_j, j = 1, \cdots, m \tag{2.10}$$

$$\sum_{i=1}^{n} h_{li} x_i \quad \leq \quad e_l, l = 1, \cdots, k \tag{2.11}$$

$$x_i \quad \in \quad \{0,1\}, i = 1, \cdots, n. \tag{2.12}$$

avec $c \in \mathbb{R}^n, b \in \mathbb{Z}^m, e \in \mathbb{Z}^k, A \in \mathbb{Z}^{n \times m}, H \in \mathbb{Z}^{n \times m}$. Admettons que l'ensemble des contraintes (2.10) est l'ensemble des contraintes difficiles. Soit $\lambda \in \mathbb{R}^m$ un vecteur de multiplicateurs Lagrangiens. Nous définissons la Relaxation Lagrangienne (RL), du problème (P), associée aux contraintes difficiles (2.10) comme suit :

ISTA

$$L(\lambda) \quad : \quad min \sum_{i=1}^{n} C_i x_i + \sum_{j=1}^{m} \lambda_j (a_{ji} x_i - b_j) \tag{2.13}$$

$$s/c$$

$$\sum_{i=1}^{n} h_{li} x_i \quad \leq \quad e_l, l = 1, \cdots, k \tag{2.14}$$

$$x_i \quad \in \quad \{0,1\}, i = 1, \cdots, n. \tag{2.15}$$

Notons que, contrairement à la programmation linéaire, la programmation linéaire en variables binaires n'a pas de dualité forte (cfr Théorème 2.2). Cela veut dire que le problème dual Lagrangien (D)

$$L_\lambda = max_{\lambda \in \mathbb{R}_+^m} L(\lambda), \tag{2.16}$$

et le problème primal initial (P) n'ont pas forcément la même valeur optimale. Selon le théorème 2.1 de dualité faible , au lieu de $L(\lambda) = Z$, nous avons alors $L(\lambda) \leq Z$. Soit x^* une solution optimale au problème primal (P). Nous observons que

$$L(\lambda) \leq \sum_{i=1}^{n} C_i x_i^* + \sum_{j=1}^{m} \lambda_j (a_{ji} x_i^* - b_j) = Z, i = 1, \cdots, m. \tag{2.17}$$

L'égalité dans cette équation provient du fait que $Z = \sum_{i=1}^{n} C_i x_i^*$ et $\sum_{j=1}^{m} \lambda_j (a_{ji} x_i^* - b_j) = 0$ pour tout $j = 1, \cdots, m$. Si les contraintes d'égalité (2.10) sont remplacées par des contraintes P d'inégalité inférieure $\sum_{j=1}^{m} \lambda_j a_{ji} x_i^* \leq b_j$ dans (P), le vecteur de multiplicateurs Lagrangiens devient positif $\lambda \in \mathbb{R}_+^m$ et nous avons

$$L(\lambda) \leq \sum_{i=1}^{n} C_i x_i^* + \sum_{j=1}^{m} \lambda_j (a_{ji} x_i^* - b_j) \leq Z, i = 1, \cdots, m. \tag{2.18}$$

où la seconde inégalité se justifie par $Z = \sum_{i=1}^{n} C_i x_i^*, \lambda_j \geq 0$ et $\sum_{j=1}^{n} a_{ji} x_i^* \leq b_j$ pour tout $j = 1, \cdots, m$. Similairement, pour les contraintes d'inégalité supérieure $\sum_{j=1}^{n} a_{ji} x_i^* \geq b_j$, le vecteur de multiplicateurs Lagrangiens devient négatif $\lambda \in \mathbb{R}_-^m$ pour avoir $L(\lambda) \leq Z$.

En général, trouver un vecteur λ pour $L(\lambda) = Z$ n'est pas garanti. Le fait que $L(\lambda) \leq Z$ nous permet d'utiliser la Relaxation Lagrangienne, au lieu de la relaxation linéaire, pour fournir des bornes inférieures (supérieures en cas de maximisation) au problème primal (P) dans l'algorithme de Branch and Bound(cfr.section....).

2.7.1 Résolution du problème dual Lagrangien

Pour simplifier, nous supposons que le problème primal est faisable et que l'ensemble $X = \{x \in \{0,1\}^n | \sum_{i=1}^{n} h_{li} x_i \leq e_l, l = 1, \cdots, k\}$ est un ensemble fini

$X = \{x^t, t = 1, \cdots, T\}$. Cela nous permet d'écrire $L(\lambda)$ de la manière suivante :

$$L(\lambda) : \sum_{i=1}^{n} C_i x_i^t + \sum_{j=1}^{m} \lambda_j (a_{ji} x_i^t - b_j), t = 1, \cdots, T; \tag{2.19}$$

qui est une fonction concave et linéaire par morceaux. La figure 1.1 montre la forme de $L(\lambda)$ pour un exemple de $m = 1$ et $T = 4$.

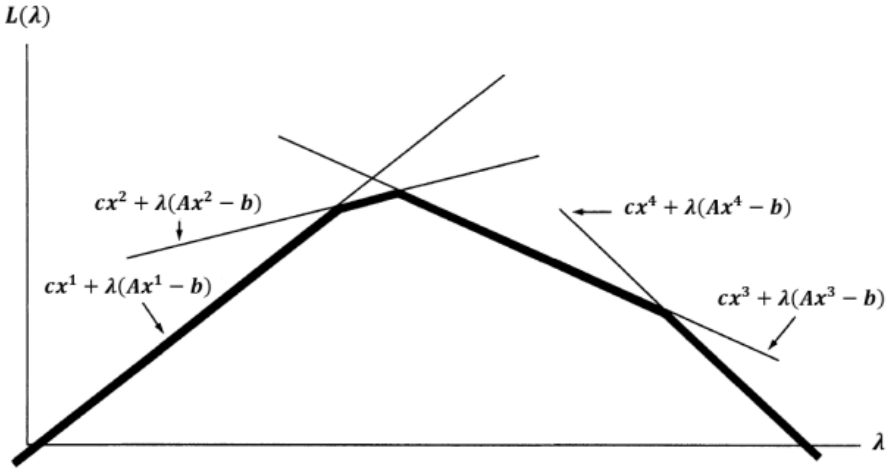


Figure 2.3: La forme de la fonction $L(\lambda)$ pour m=1 et T=4

.

La fonction $L(\lambda)$ n'est pas différentiable partout. En conséquence, nous ne pouvons pas appliquer directement la méthode du gradient (voir section 1.8) pour maximiser les bornes inférieures obtenues, mais plutôt la méthode du sous-gradient (les gradients sont remplacés par des sous-gradients).

Definition 2.1 *Nous appelons $y \in \mathbb{R}_+^m$ un sous-gradient de $L(\lambda)$ au $\bar{\lambda}$ s'il satisfait*

$$L(\lambda) \leq L(\bar{\lambda}) + y(\lambda - \bar{\lambda}). \tag{2.20}$$

Soit $\lambda_0 \in \mathbb{R}_+^m$ une valeur initiale. La séquence $\{\lambda\}$ est générée selon la règle

$$\lambda^{k+1} = \lambda^k + t_k(Ax^k - b), \tag{2.21}$$

où $Ax^k - b$ est un sous-gradient au λ^k pour lequel x^k est une solution optimale au problème Lagrangien relaxé (RL) (équations (2.9)-(2.12) et la valeur t^k représente la taille du pas. La solution donc optimale $L(\lambda^k)$ est trouvée lorsque $t^k \to 0$ et $\sum_{i=1}^{k} t_i \to \infty$. En pratique, la taille du pas est souvent calculée par la formulation suivante :

$$t^k = \lambda^k \frac{(Z^* - L(\lambda_k))}{\|Ax^k - b\|^2}, \tag{2.22}$$

ISTA

où $0 < \lambda_k \leq 2$ est un scalaire et Z^* est la borne supérieure de L_λ(cfr Equation (2.16)). Notons que la méthode du sous-gradient n'a aucun moyen pour prouver l'optimum, l'algorithme s'arrête alors lorsqu'il atteint un nombre maximum d'itérations fixé préalablement.

2.8 Exemple

Soit le problème linéaire en variables binaires suivant

$$\begin{aligned} Z &: max\ 16x_1 + 10x_2 + 4x_3 && (2.23)\\ s/c & \\ 8x_1 &+ 2x_2 + x_3 + 4x_4 \leq 10 && (2.24)\\ x_1 &+ x_2 \leq 1 && (2.25)\\ x_3 &+ x_4 \leq 1 && (2.26)\\ x &\in \{0,1\}^4, . && (2.27) \end{aligned}$$

En relâchant la contrainte (2.24), nous obtenons le problème Lagrangien relaxé suivant

$$\begin{aligned} L(\lambda) = max(16 - 8\lambda)x_1 &+ (10 - 2\lambda)x_2 + (0 - \lambda)x_3 + (4 - 4\lambda)x_4 + 10\lambda && (2.28)\\ s.c. & \\ x_1 + x_2 &\leq 1 && (2.29)\\ x_3 + x_4 &\leq 1 && (2.30)\\ x \in \{0,1\}^4 & && (2.31) \end{aligned}$$

où $\lambda \in \mathbb{R}_+$ est un multiplicateur lagrangien. Trouver la meilleure valeur de λ revient à résoudre le problème dual Lagrangien (le minimum des bornes supérieures)

$$L(\lambda) = min_{\lambda \geq 0} L(\lambda). \quad (2.32)$$

Avant d'appliquer un algorithme de résolution à ce problème (2.32), nous allons tester différentes valeurs de λ. Le tableau suivant liste sept valeurs de λ, avec la solution au problème Lagrangien relaxé associé, la borne supérieure $L(\lambda)$ et la valeur Z pour les solutions faisables dans le problème primal.

Selon le tableau ci-dessus, la meilleure borne supérieure $(L(\lambda) = 18)$ est obtenue avec $\lambda = 1$. Ce cas présente quatre alternatives de solutions au problème Lagrangien relaxé : trois solutions faisables $(Z = 10, 14, 16)$ pour le problème primal et une solution infaisable. Cette borne $(L(\lambda) = 18)$ représente la solution optimale au problème Lagrangien relaxé. Pour le montrer, il suffit

ISTA

λ	x_1	x_2	x_3	x_4	$L(\lambda)$	Z
0	1	0	0	1	20	
6	0	0	0	0	60	0
3	0	1	0	0	34	10
2	0	1	0	0	26	10
1	1	0	0	0	18	16
1	1	0	0	1	18	
1	0	1	0	0	18	10
1	0	1	0	1	18	14
1/2	1	0	0	1	19	
3/4	1	0	0	1	18.5	

Figure 2.4: Solutions obtenues par la Relaxation Lagrangienne pour différentes valeurs de λ

d'observer que quelque soit la valeur de x, nous obtenons toujours une fonctions linéaire en λ. La figure ci dessous présente cette famille de fonctions linéaires pour toutes les solutions optimales au problèmes Lagrangien relaxé. Comme l'objectif Lagran- gien est une maximisation, la fonction $L(\lambda)$ est alors définie sur l'enveloppe supérieur de ces fonctions linéaires, sous forme d'une fonction linéaire par morceaux (cfr Figure suivante)

ISTA

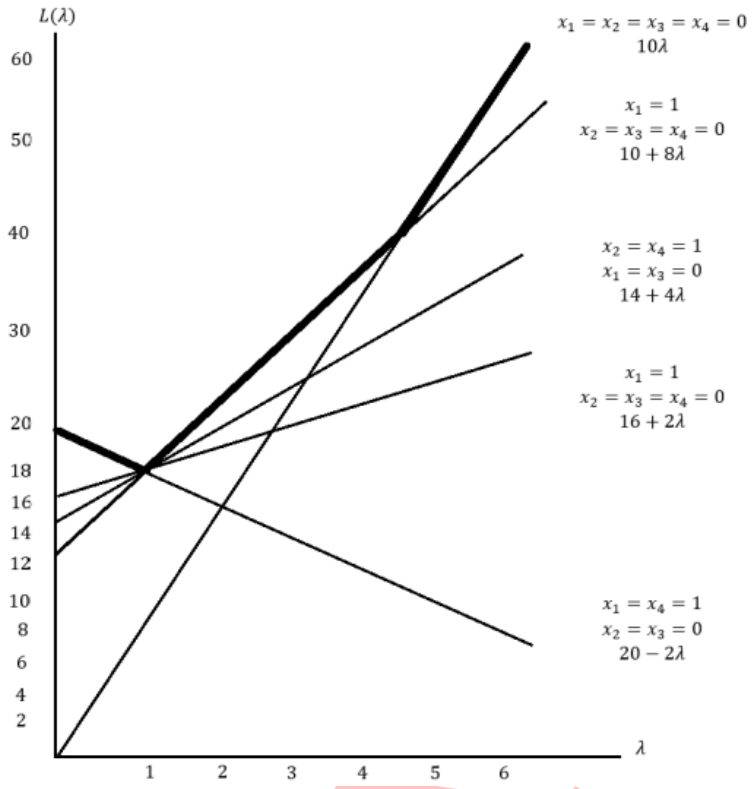


Figure 2.5: La fonction $L(\lambda)$

Comme mentionné auparavant, la fonction $L(\lambda)$ est une fonction convexe et sous- différentiable. Elle n'est pas différentiable aux points où le problème Lagrangien relaxé a de multiples solutions (Par exemple, le point $\lambda = 1, L(\lambda) = 18$ sur la figure 2.5. A ces points, la dérivée de $L(\lambda)$ est définie comme suit $8x_1+2x_2+x_3+4x_4-10$, où x est une solution optimale au problème Lagrangien relaxé. Donc, pour minimiser $L(\lambda)$, i.e. résoudre le problème (2.32), nous appliquons la méthode du sous-gradient. Pour résumer, la méthode consiste à trouver une séquence de valeurs de λ en commençant avec une valeur initiale λ_0 et en appliquant la formule suivante

$$\lambda^{k+1} = max\{0, \lambda^k - t^k(b - At^k)\}. \tag{2.33}$$

où t_k est la taille du pas et x_k la solution optimale du problème Lagrangien relaxé. Le tableau 2.4 présente les résultats de la méthode du sous-gradient sur l'exemple correspondant à trois procédures de réglage de t_k . Dans la première colonne, t_k est fixé à 1 pour toutes les itérations et nous remarquons que les résultats oscillent entre deux valeurs $\lambda = 0$ et $\lambda = 2$. Pour la deuxième colonne, t_k converge vers 0 avec un pas égal à 1/2 et les résultats convergent vers la solution optimale $\lambda = 1$. Dans la troisième colonne, t_k converge également vers 0 mais d'une manière rapide avec un pas égal à 1/3. Dans ce dernier cas, nous remarquons que λ converge vers 1/3 au lieu de converger vers 1. Nous pouvons retenir alors que si la taille du pas t_k converge rapidement vers 0, la méthode du sous-gradient pourra s'éloigner de

ISTA

$\lambda^0 = 0$	$\lambda^0 = 0$	$\lambda^0 = 0$
$\lambda^1 = 0-(-2) = 2$	$\lambda^1 = 0-(-2) = 2$	$\lambda^1 = 2$
$\lambda^2 = max\{0, 2-8\} = 0$	$\lambda^2 = max\{0, 2-1/2(8)\} = 0$	$\lambda^2 = max\{0, 2-1/3(8)\} = 0$
$\lambda^3 = 0-(-2) = 2$	$\lambda^3 = 0-1/4(-2) = 1/2$	$\lambda^3 = 0-1/9(-2) = 2/9$
$\lambda^4 = max\{0, 2-8\} = 0$	$\lambda^4 = 1/2-1/8(-2) = 3/4$	$\lambda^4 = 2/9-1/27(-2) = 0.296$
	$\lambda^5 = 3/4-1/16(-2) = 7/8$	$\lambda^5 = 0.296-1/81(-2) = 0.321$
	$\lambda^6 = 7/8-1/32(-2) = 15/16$	$\lambda^6 = 0.321-1-243(-2) = 0.329$
		$\lambda^7 = 0.329-1/729(-2) = 0.332$

Figure 2.6: Les résultats de la méthode du sous-gradient, sur l'exemple, obtenus avec $t^k = 1$ pour la première colonne, $t^k = 1, 1/2, 1/4, 1/8, \cdots$ pour la deuxième colonne et $t^k = 1, 1/3, 1/9, 1/27, 1/81, \cdots$ pour la troisième colonne

l'optimum. Une condition nécessaire mais non suffisante pour que $L(\lambda)$ tend vers l'optimum est $\sum_{i=1}^{k} t_i \to \infty$. Cette condition est violée dans le troisième cas ($\sum_{i=1}^{k} t_i \to 2$) où le résultat s'éloigne de l'optimum ($\lambda = 1$).

Comme cité avant, la méthode du sous-gradient ne garantie pas d'avoir l'optimum. Dans cet exemple, nous avons obtenu une solution égale à $Z = 16$ et une borne supérieure égale à $L(\lambda = 1) = 18$. Pour prouver la meilleure solution entière ($Z^* = 16$), il faut continuer la recherche en appliquant la méthode arborescente Branch and Bound (avec des bornes fournies par la relaxation Lagrangienne), qui est coûteuse en terme de temps de calcul pour les problèmes de grande taille.

2.8.1 Relaxation Lagrangienne vs. relaxation linéaire

Pour comparer les bornes obtenues par la relaxation Lagrangienne avec celles fournies par la relaxation linéaire, nous considérons le problème linéaire ((2.23)-(2.27)) montré dans l'exemple précédent. Son problème dual linéaire s'écrit de la manière suivante :

$$
\begin{aligned}
min\ & 10\lambda + v_1 + v_2 + w_1 + w_2 + w_3 + w_4 \qquad (2.34)\\
s.c.\ & \\
& 8\lambda + v_1 + w_1 \geq 16\\
& 2\lambda + v_1 + w_2 \geq 10\\
& \lambda + v_2 + w_3 \geq 0\\
& 4\lambda + v_2 + w_4 \geq 4\\
& \lambda \quad , \quad v_1, v_2, w_1, \cdots, w_4 \geq 0
\end{aligned}
$$

où λ, v_1, v_2 représentent respectivement les variables duales des contraintes (2.24), (2.25), (2.26) et w_j les variables duales des contraintes $x_j \leq 1$ (2.27).

La solution optimale au problème primal est $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1/2$ (n'est pas entière) et la solution optimale au problème dual est $\lambda = 1, v_1 =$

$8, v_2 = w_1 = \cdots = w_4 = 0$. Les deux solutions donnent une valeur objectif égale à $Z_{LP} = 18$.

Nous avons donc obtenu, avec la relaxation linéaire ($Z_{LP} = 18$), la même borne supérieure trouvée par la relaxation Lagrangienne. De plus, la variable duale $\lambda = 1$ de la contrainte (2.10) a exactement la même valeur trouvée par le problème dual Lagrangien. Cela est justifié par un ensemble de relations entre les problèmes d'optimisation. Nous avons :

$$\begin{aligned} L_\lambda &= min\{max\{cx + \lambda(b - Ax) : Hx \leq e, x \geq 0 \; et \; entier\}\} \\ &\leq min\{max\{cx + \lambda(b - Ax) : Hx \leq e, x \geq 0\}\}. \end{aligned} \tag{2.35}$$

Soit v les variables duales associées aux contraintes $Hx \leq e$. En remplaçant (deux fois) le problème primal par son dual, nous obtenons:

$$\begin{aligned} L_\lambda &\leq min\{min\{\lambda bve : \lambda A + vH \geq c, \lambda, v \geq 0\}\} \\ &= max\{cx : Ax \leq b : Hx \leq e, x \geq 0\}. \end{aligned} \tag{2.36}$$

Ces relations montrent que si le problème Lagrangien ne change pas après la suppression des contraintes d'intégrité, alors $L_\lambda = Z_{LP}$. La relaxation Lagrangienne donne toujours des valeurs entières aux variables que ce soit avec ou sans contraintes d'intégrité. Ce qui implique le résultat $L_\lambda = Z_{LP} = 18$.

Nous pouvons donc améliorer la valeur de la borne supérieure en utilisant la relaxation Lagrangienne qui a l'avantage (par rapport à la relaxation linéaire) de donner directement des solutions entières. Dans ce sens, nous présentons une méthode alternative de relaxation Lagrangienne de l'exemple

$$\begin{aligned} L(v_1, v_2) = max(16 - v_1)x_1 &+ (10 - v_1)x_2 + (0 - v_2)x_3 + (4 - v_2)x_4 \\ &+ v_1 + v_2 \end{aligned} \tag{2.37}$$

$$\begin{aligned} s.c. & \\ 8x_1 &+ 2x_2 + x_3 + 4x_4 \leq 10 \\ x &\in \{0, 1\}^4. \end{aligned} \tag{2.38}$$

où nous relaxons les contraintes (2.25) et (2.26) à l'aide des variables duales $v_1 \geq 0$ et $v_2 \geq 0$. Nous obtenons alors le problème classique du sac à dos.Donc,l'application de la méthode du sous-gradient trouve une solution duale $(x_1 = 1, x_2 = x_3 = x_4 = 0)$ pour laquelle la borne supérieure $(L(v_1, v_2) = 16)$ est égale à la solution du problème primal $(Z^* = 16)$.

2.8.2 Applications de la relaxation Lagrangienne

De nombreux problèmes combinatoires ont été résolus de façon exacte ou approchée par l'emploi de la relaxation Lagrangienne, à savoir le problème

d'affectation généralisée , le problème du voyageur de commerce et le problème de plus court chemin avec contrainte. Dans la littérature, un grand nombre d'auteurs utilisent des heuristiques Lagrangiennes vu que la relaxation Lagrangienne basique ne garantit pas d'avoir une solution faisable au problème primal. Ces heuristiques sont en majorité utilisées dans une approche de Branch and Bound qui permet de trouver des solutions optimales. Nous citons par exemple [Holmberg and Yuan, 2000] qui proposent une heuristique Lagrangienne intégrée dans un schéma de Branch and Bound pour la résolution du problème de conception de réseau (network design). Or pour des problèmes de grandes tailles, ces méthodes arborescentes deviennent coûteuses en terme de temps de calcul. La solution est donc d'appliquer les heuristiques Lagrangiennes dans un outil de résolution pour fournir des solutions approchées de bonnes qualités en temps raisonnable. Dans ce sens, [Seki et al., 2010] proposent la méthode de recherche à voisinage pour améliorer les solutions obtenues par la Relaxation Lagrangienne et l'appliquent sur le problème de planification d'unités de production électrique (Unit Commitment Problem). [Klincewicz and Luss, 1986] décrivent une heuristique basée sur la relaxation Lagrangienne qui utilise la méthode de descente avec une application sur le problème de localisation d'entrepôts avec capacité (Capacitated Facility Location with Single- Source Constraints). [Xie and Turnquist, 2011] proposent une méthode hybride pour les problèmes d'optimisation des réseaux d'évacuation. La méthode combine la relaxation lagrangienne avec la recherche Tabou, la première décompose le problème en sous-problèmes Lagrangiens et la seconde résout ces derniers et adapte les multiplicateurs de Lagrange.

2.9 Méthodes de résolution

Un grand nombre de méthodes de résolution existe en recherche opérationnelle pour l'optimisation combinatoire. Nous distinguons deux grandes familles: les méthodes exactes qui garantissent la complétude de la résolution (optimalité) à des temps de calcul parfois prohibitifs ; et les méthodes approchées qui peuvent perdre la complétude de résolution mais qui fournissent souvent des solutions approchées de bonne qualité en temps raisonnable.

2.9.1 Méthodes exactes

Les méthodes exactes sont des méthodes de résolution qui permettent d'obtenir la solution optimale à un problème d'optimisation en parcourant, de manière implicite, toutes les combinaisons possibles.

Dans cette famille, nous citons les méthodes de séparation et évaluation (Branch and Bound) [Land and Doig, 1960]. Leur principe de base consiste

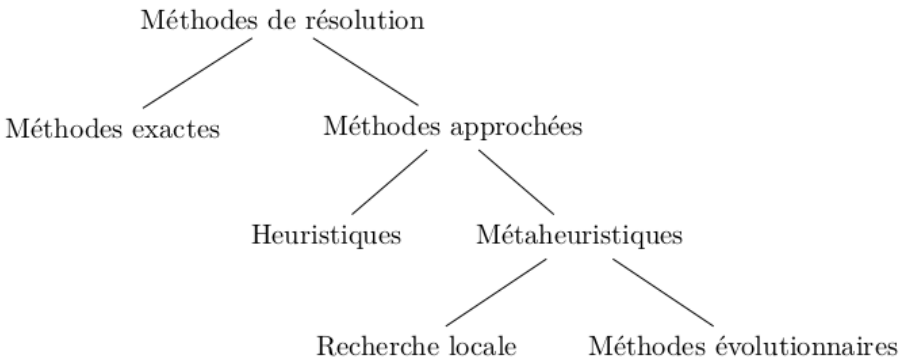


Figure 2.7: Classification des méthodes d'optimisation

à diviser le problème initial en un certain nombre de sous-problèmes pour constituer l'arbre de recherche. A chaque itération, un sous-problème est sélectionné 1) pour déterminer l'optimum de l'ensemble des solutions réalisables associées au sous-problème en question, ou 2) pour prouver que cet ensemble ne contient pas de solution optimale. La méthode la plus générale pour prouver l'optimum consiste à déterminer un minorant (majorant pour une maximisation) des solutions contenues dans l'ensemble, en résolvant une relaxation du sous-problème courant. Si le minorant est supérieur à la valeur de la meilleure solution trouvée, cela affirme que le sous-ensemble ne contient pas l'optimum. Par contre, si les deux bornes supérieure et inférieure sont égales, alors une solution optimale est trouvée.

Plusieurs améliorations de l'algorithme de Branch and Bound ont été proposées. Nous citons la méthode de Branch and Cut [Mitchell, 2011] qui utilise, en plus de la méthode de séparation et évaluation, la méthode des plans sécants. L'objectif de cette dernière consiste à ajouter des contraintes au programme linéaire pour le raffiner et le rapprocher des so- lutions intégrales. Elle est appliquée au niveau de chaque noeud de l'arbre de recherche lorsque la résolution du problème relaxé donne une solution fractionnaire.

Comme inconvénient majeur, ces méthodes exactes d'énumération sont parfois caractérisées par un temps de calcul prohibitif. Pour réduire le temps d'évaluation d'un noeud et améliorer son efficacité, des méthodes de décomposition sont utilisées, comme la décomposition de Benders [Benders, 1962] et la décomposition de Dantzig-Wolfe [Dantzig and Wolfe, 1960]. Les problèmes décomposés avec ce dernier type utilisent dans leurs résolutions la méthode de génération de colonnes, appelée Branch and Price [Barnhart et al., 1998] lorsqu'elle est combinée avec la méthode de Branch and Bound. L'idée centrale de la méthode de génération de colonnes est de commencer la résolution, d'un problème linéaire de grande taille, avec un sous-ensemble de colonnes de petite taille. Puis de générer, au sein d'un algorithme à plusieurs étapes, les variables (colonnes) qui sont susceptibles d'améliorer la solution courante.

En général, le principe de décomposition est le même pour toutes les

méthodes de dé- composition : le problème initial est décomposé en un ou plusieurs sous-problèmes faciles, généralement coordonnés par un problème maître.

Nous présentons la décomposition de Benders [Benders, 1962], qui est une méthode itérative alternant entre la résolution d'un problème maître et la résolution d'un sous- problème. Le problème maître est en général plus facile à résoudre que le problème original car il s'agit d'une relaxation. Le sous-problème génère donc des coupes pour renforcer le problème maître jusqu'à ce que ce dernier soit équivalent au problème initial.

Considérons le problème mixte suivant

$$minc^T x + d^T z \tag{2.39}$$

$$sc$$

$$AX + A'z \leq z \tag{2.40}$$

$$x \in \{0,1\}^n \tag{2.41}$$

$$z \in \mathbb{R}^p_+ \tag{2.42}$$

Pour un vecteur fixe $x \in \{0,1\}^n$, nous pouvons reformuler le problème de la manière suivante :

$$min_{\bar{x}\in\{0,1\}^n}\{c^T\bar{x} + \min_{z\geq 0}\{d^T z : A'z \leq b - A\bar{x}\}\}. \tag{2.43}$$

La minimisation à l'intérieur de l'équation (2.47) (le sous-problème) est un problème linéaire continu, que nous pouvons dualiser en introduisant un vecteur de variables duales $\pi \in \mathbb{R}^m$ associé à l'ensemble des contraintes $A'z \leq b - A\bar{x}$

$$max_{\pi\in\mathbb{R}^m}\{\pi^T(b - A\bar{x}) : \pi^T A' \leq d\} \tag{2.44}$$

A partir des deux équations (2.47) et (2.44), nous pouvons déduire la formulation suivante :

$$min_{\bar{x}\in\{0,1\}^n}\{c^T\bar{x} + \max_{\pi\in\mathbb{R}^m}\{\pi^T(b - A\bar{x}) : \pi^T A' \leq d\}\}. \tag{2.45}$$

L'espace des solutions admissibles du sous-problème dual (2.44), i.e., $F = \{\pi|\pi^T A' \leq d\}$ ne dépend pas du choix de $\bar{x}$. Donc, si F est un ensemble non vide, le sous-problème est donc soit non borné ou réalisable quelle que soit la valeur de $\bar{x}$. En effet, si le sous-problème dual (2.44) est non borné, cela implique que le sous-problème primal (2.47) est infaisable. Pour éviter

ISTA

les mouvements dans la direction non bornée, nous ajoutons au problème maître une coupe de faisabilité

$$r_q^T(b - A\bar{x}) < 0 \forall q \in Q \tag{2.46}$$

où $r_q, q \in Q$, est un rayon extrême non borné, Q est l'ensemble des rayons extrêmes. Or, si le sous-problème dual est réalisable, sa solution est donc un des points extrêmes $\pi_e, e \in E$, avec E est l'ensemble des points extrêmes. Le problème (2.47) devient :

$$min_{\bar{x}\in\{0,1\}^n} c^T\bar{x} + \max_{\pi\in\mathbb{R}^m}\{\pi^T(b - A\bar{x})\}. \tag{2.47}$$

$$sc$$

$$r_q^T(b - A\bar{x}) < 0 \forall q \in Q. \tag{2.48}$$

Ainsi, en introduisant une variable continue $\eta \in \mathbb{R}$, la résolution du problème initial (2.39)-(2.42) devient équivalent à résoudre :

$$min_{x,\eta} c^T x = \eta \tag{2.49}$$

$$sc$$

$$\eta \geq \pi_e^T(b - Ax) < 0 \forall e \in E \tag{2.50}$$

$$0 \geq r_q^T(b - A\bar{x}) < 0 \forall q \in Q, \tag{2.51}$$

$$x \in \{0, 1\}^n \tag{2.52}$$

L'algorithme de décomposition de Benders est une approche itérative, il n'énumère pas toutes les coupes d'optimalité (2.50) et de faisabilité (2.51). Il résout le problème maître avec un sous-ensemble de coupes afin d'obtenir une valeur pour la variable x, i.e., x, puis il résout le sous-problème (2.44) avec cette valeur x. Si le sous-problème est faisable et borné, l'algorithme produit une coupe de type (2.50). Si le sous-problème est non borné, une coupe de type (2.51) est produite. Ensuite, si ces coupes sont violées par la solution courante, l'algorithme les insère directement dans le problème maître (voir Figure 2.8. La procédure se répète jusqu'à l'obtention d'une solution optimale (la solution du problème maître et celle du sous-problème sont égales).

Cet algorithme de décomposition de Benders est implémenté dans le solveur IBM ILOG cplex (voir la sous-section 1.8.3), à partir de sa version 12.7.0 [CPLEX, 2015]. Toutefois, malgré les progrès réalisés, les méthodes exactes deviennent inefficaces à mesure que la taille des problèmes devient importante, d'où la nécessité d'appliquer des méthodes approchées.

2.10 Méthodes approchées

Les méthodes approchées représentent une alternative pour résoudre les problèmes d'opti- misation de grande taille lorsque les méthodes exactes échouent. Elles permettent d'obtenir des solutions de bonne qualité en un temps de calcul réduit. Elles sont fondées principa- lement sur des heuristiques, qui peuvent être spécifiques à un type de problème, telle que la méthode de Wedelin (voir partie II) ou générales. Cette dernière catégorie appelée également métaheuristique est subdivisée en deux grandes familles : 1) les méthodes de recherche locale [Aarts et al., 2003] à solution unique telle que la méthode de descente, le Recuit Simulé et la recherche Tabou ; et 2) les méthodes évolutionnaires [Coello, 2005] à base de population de solutions comme les algorithmes génétiques (voir figure 2.7).

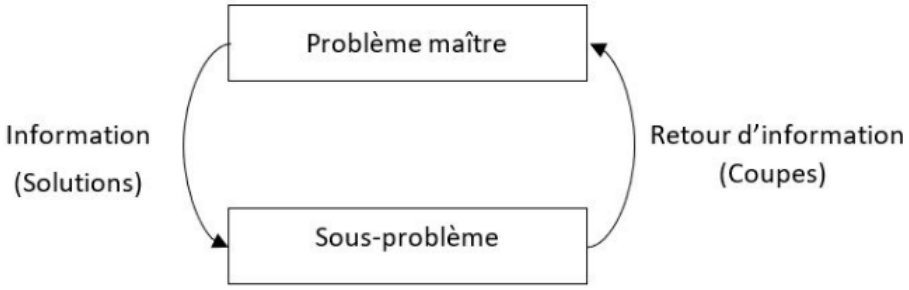


Figure 2.8: TIllustration d'une itération de l'algorithme de décomposition de Benders.

Méthode de descente (Hill Climbing)

La méthode de descente est une méthode classique de la recherche locale. Son principe général est le suivant : i) débuter avec une solution initiale x dont la valeur objectif est $f(x)$, ii) choisir une solution voisine x' de x telle que $f(x') < f(x)$, puis remplacer x par x' et répéter ii) jusqu'à ce que pour tout voisin x' de x, $f(x) \leq f(x')$. x est alors un optimum local. Pour trouver l'optimum global, une version améliorée de l'algorithme de descente (Iterated Local Search, ILS) consiste à changer la direction de recherche lorsqu'un optimum local est trouvé, en repartant d'une nouvelle solution générée aléatoirement.

Recuit simulé

Le recuit simulé est une métaheuristique inventée par les physiciens Kirkpatrick, Gellat et Vecchi en 1983 [Kirkpatrick et al., 1983]. Elle est inspirée

du recuit des métaux en métal- lurgie dont le principe est d'alterner des cycles de refroidissement lent et de réchauffage. A la base, le refroidissement naturel de certains métaux ne permet pas aux atomes de se placer dans une configuration la plus solide. Lorsque le métal est réchauffé, l'énergie disponible rend aux atomes leur mobilité. Si le métal est refroidi lentement, la mobilité thermique diminue et les atomes cherchent à former de nouvelles liaisons. Physiquement, le mécanisme de minimisation de l'énergie repose sur la distribution de probabilité de Boltzmann $p(E) = exp(-E/kT)$ où T est la température à l'équilibre thermique, k est une constante de Boltzmann et E représente l'état d'énergie. L'objectif final de ce processus est de parvenir, d'un état méta-stable correspondant à un minimum local d'énergie, à l'état stable de plus basse énergie. La probabilité qu'un système physique passe d'un niveau d'énergie E_1 à un niveau E_2 est donnée par la fonction $p = exp[-(E_1 - E_2)/kT]$, qui est toujours inférieure à 1 si $E_1 \leq E_2$. Pour résumer, la méthode du recuit simulé considère un processus d'exploitation du voisinage, qui permet de se diriger vers une solution voisine de moins bonne qualité avec une probabilité non nulle pour éviter les optima locaux. L'algorithme commence avec 1) une température initiale $T = T_0$ qui décroît tout au long de l'algorithme pour tendre vers 0 et 2) une probabilité d'acceptation (des solutions moins bonnes) élevée. Plus la température T décroît, plus la probabilité p diminue. La difficulté du recuit simulé réside dans le choix de ses paramètres : la température initiale et la loi de décroissance de la température. Cette méthode peut nécessiter un critère d'arrêt lorsque la probabilité d'acceptation devient très faible ou lorsque la fonction d'évaluation cesse d'évoluer.

Recherche Tabou

La recherche Tabou est une méthode heuristique présentée par Glover [Glover, 1990]. Son principe de base consiste à poursuivre la recherche, même lorsqu'un optimum local est trouvé, en utilisant une notion de mémoire. Pour éviter les mouvements cycliques entre un optimum local et son voisin, la méthode considère une liste taboue contenant des mouvements ou des solutions qui sont temporairement interdits. La taille de cette liste, généralement gérée en FIFO (First In First Out), est un paramètre ajustable de l'heuristique. Également, cette stratégie nécessite de définir un critère d'arrêt (nombre d'itérations, temps de calcul, solution prouvée optimale, ...).

Algorithmes génétiques

Les algorithmes génétiques [Coello, 2005] sont des algorithmes d'optimisation fondés sur les mécanismes de la génétique. Ils simulent le processus d'évolution d'une population, où chaque individu représente une solution x. Le degré

d'adaptation d'un individu à l'environnement est exprimé par une fonction de fitness $f(x)$. Le fonctionnement est simple. Nous partons d'une population de N individus créés aléatoirement. Chaque individu de la population est évalué par la fonction objectif. Ensuite, une stratégie de sélection choisit des individus pour former une population parent. Sur les individus de cette dernière, un opérateur de croisement est appliqué pour obtenir une population enfant, puis un opé- rateur de mutation est appliqué sur les individus de la population enfant. La nouvelle population obtenue par le choix de N individus parmi les populations parent et enfant est appelée génération suivante. De cette manière, nous produisons une population plus riche en individus qui sont mieux adaptés aux contraintes du problème. La figure 1.5 illustre le schéma général d'un algorithme génétique, où chaque itération représente une génération.

ISTA

Chapter 3

Notions de base en complexité des algorithmes

Ce chapitre vise à fournir des résultats de la théorie de la complexité qui aident à comprendre un certain nombre de phénomènes rencontrés en optimisation combinatoire et en particulier la frontière qui sépare problèmes faciles et difficiles.

ISTA

3.1 Rappel d'algorithmique

Definition 3.1 *(Il y en a plusieurs) Un algorithme peut être défini comme:*

- *Une suite d'opérations (d'actions, d'ordres, d'instructions);*
- *Des structures de contrôle (SDC) manipulant des structures de données (SDD);*
- *Un ensemble d'objets ayant des attributs et des opérations échangeant des messages;*
- *D'une manière générale: une représentation automatisable d'une méthode;*
  . . . dont l'implémentation permet de résoudre un certain problème par une machine.

SDC

(a) Affectation :

- Variable = Expression ;
- Lecture ;

- Ecriture;

(b) Condition :

- Simple (if simple);
- avec alternative (if avec else, elseif);
- multi-choix (switch);

(c) Itération :

- Un nombre connu d'itérations (for) ;
- Un nombre d'itérations inconnu (while) ;
- Au moins une itération (do ... loop until) ;

(d) Récursion (self appel) :

- Simple : un seul self appel ;
- Multiple : plusieurs self appels ;
- Mutuelle : appels mutuels ;
- Terminale : l'appel récursif est la dernière action ;

Remark 3.1 *Toutes ces structures pourraient être en séquence et/ou en parallèle.*

SDD

a. Concrètes (physiques)

- Constante, variable simple (bit, boolean, byte, char, int, float , double, string, ...) ;
- Tableau (uni ou multi dimensionnel) ;
- Liste chainée (à l'aide de pointeurs) ;

b. Abstraites (implémentées par des SDD concrètes):

- Linéaires (Ensemble, Liste, Pile, File) ;
- Arborescentes (Arbre, Graphe) ;

ISTA

Quelques exemples d'algorithmes

Calcul du PGCD de deux entiers

```
Int gcd (int a, int b)
    {  while (a<>b)
        if (a>b) a-=b ;
        else b-=a ;
     Return   a}
```

Recherche d'un entier dans un tableau d'entiers

```
Recherche (int[] A, int n, int x)
    {  for (i=0 ; i<n ; i++)
        If (A[i]==X) return i ;
     Return   false}
```

Calcul du minimum d'un tableau d'entiers

```
Min (int[] A, int n, )
    {  M = A[0] ; ind = 0 ;
for (i=0 ; i<n ; i++)
        If (A[i]<M) { M = A[i] ; ind = i ;} ;
     Return   ind}
```

Tri d'un tableau d'entiers par insertion

```
tri_insertion(tableau A, n)
    {  for (i=1 ; i<n ; i++)
        {x = A[i]
        j = i
        while (j > 0 and A[j - 1] > x)
              {  T[j] = A[j - 1] ;
               j - - ;}
        T[j] ← x } }
```

Propriétés d'un algorithme

Pour qu'il soit valide, un algorithme doit satisfaire trois propriétés:

- Terminaison (il faut qu'il se termine) ;
- Correction (il faut qu'il soit conforme à la spécification du problème à résoudre) ;

- Complétude (doit solutionner toutes les instances du problème en question);
  Sinon, l'algorithme n'est pas valide.

Types d'algorithmes

(Il y en a plusieurs classifications)

a. Selon le mode d'exécution :

- Séquentiel: ses actions s'exécutent séquentiellement(l'une après l'autre);
- Parallèle : certaines de ses actions s'exécutent en parallèle (en même temps sur un ordinateur multiprocesseur ou avec un seul processeur mais multicœurs) ;
- Distribué : certaines de ses actions s'exécutent en parallèle (en même temps sur un réseau d'ordinateurs) ;

b. Selon la qualité de la solution fournie :

- Exact (optimal) : il fournit une solution exacte (prouvée formellement) qui répond exactement au problème.

  Example 3.1

  I. L'algorithme de résolution d'une équation du second degré par la méthode du discriminant) ;

  II. Le tri par bulles ;

  III. L'algorithme de Bellman calculant le plus court chemin dans un graphe.

  Approche: il fournit une solution approchée (prouvée empiriquement) qui répond approximativement au problème.

  I . L'algorithme de résolution d'une équation du troisième degré par le théorème des valeurs intermédiaires);

  II. L'algorithme de calcul d'une valeur approchée de $\pi, e^x, \sqrt{x}, \cdots$ par le développement limité de Taylor ou autre méthode ;

c. Selon le déterminisme de ses actions :

- Déterministe : pas d'actions probabilistes (soumises au hasard).
- Indéterministe (stochastique) : l'algorithme comporte des actions probabilistes (soumises au Hazard).

ISTA

Example 3.2 *Algorithme d'un jeu, if (rand(0,1) < 0.5) print ('face') else print ('pile') ;*

Remark 3.2 *Tous les jeux sont des algorithmes stochastiques.*

d. Selon le mode des actions répétitives :

- Itératif : l'algorithme n'appelle (n'utilise) pas lui-même (directement ou indirectement).
- Récursif : l'algorithme appelle lui-même (directement ou indirectement). On distingue : la récursivité :

∗ Simple : un seul appel récursif : calcul du factoriel d'un entier. int fact(int n) if(n==0 or n==1) return 1 ; else return n*fact(n-1);

∗ Multiple : plusieurs appels récursifs : tri fusion, calcul du nième terme de la suite de Fibonacci (récursivité double). int Fibo(int n) if(n==0 or n==1) return n ; return Fibo(n-1)+Fibo(n-2);

∗ Mutuelle : des appels mutuels : calcul de la liste des nombres premiers inférieurs à n. Bool Prem(int n) for each i in ListePrem(sqrt(n)) if (n % i ==0) return 0; return 1; Int[] ListPrem(int n) l=null; for (j=2; j<=n ; j++) if (Prem(j)) l.append(j);

Remark 3.3

Elaborer un programme récursif est beaucoup plus aisé qu'élaborer un programme itératif équivalent.

On peut toujours passer d'un programme itératif à un programme récursif équivalent et vice-versa. Cette dernière opération est dite élimination de la récursivité.

Sachant que tout programme, pour être exécuté, il doit être traduit en langage machine et puisque, dans tous les systèmes d'exploitation et toutes les architectures hardware, ce dernier n'est jamais récursif, il est donc toujours recommandé d'éviter la récursivité. Au fait, c'est le compilateur qui se charge d'éliminer la récursivité à votre place. Ainsi, éviter la récursivité veut dire éviter d'éventuelles erreurs sémantiques très difficile à détecter dans certains cas.

En effet, si on opte pour un programme récursif, il faut toujours mettre l'appel récursif comme dernière action. Dans ce cas on dit que la récursivité est terminale.

Example 3.3

I. *Récursivité non terminale : int fact(int n) if(n==0 or n==1) return 1 ; else return fact(n-1)*n; int fact(int n) if(n==0 or n==1) return 1 ; else return n*fact(n-1);*

(e) Selon le mode d'introduction des données :

Online : les données ne sont connues qu'au moment de l'exécution de l'algorithme.

Offline : les données sont disponibles avant l'exécution de l'algorithme. f) Selon la complexité

Polynomial : le nombre de ses opérations est une fonction polynomiale de la taille de son input.

Exponentiel : le nombre de ses opérations est une fonction exponentielle de la taille de son input.

Pseudo-polynomial : le nombre de ses opérations est une fonction polynomiale valeurs de son input.

3.2 Paradigmes d'algorithmique

Ce sont les stratégies, styles de conception des algorithmes pour lesquels on opte lors du design d'un algorithme. Ce choix dépend de plusieurs facteurs : la complexité du problème, la taille de son input, l'environnement à utiliser pour l'implémentation,...

- Force brute : on utilise la définition littéralement comme la formule $C_n^k = \frac{n!}{k!(n-k)!}$
- Exhaustif: énumérer tous les cas possibles

Example 3.4

I. Pour trier un tableau, on teste toutes les permutations possibles des éléments du tableau ;

II. Pour calculer le plus court chemin entre a et b, on explore tous les chemins existants entre a et b.

- Diviser pour régner : diviser le problème en sous-problèmes de tailles inferieures, résoudre les sous problèmes (en général, récursivement) puis fusionner leurs solutions pour dégager la solution du problème entier.

Example 3.5

I. Recherche dichotomique ;

II. Tri fusion ;

III. Utiliser la formule: $C_n^k = C_{n-1}^k + C_{n-1}^{k-1}$. pour calculer récursivement C_n^k .

ISTA

Calcul du nième terme de la suite de Fibonacci récursivement.

- Programmation dynamique : un cas particulier du "diviser pour régner" où les sous problèmes sont imbriqués dont les solutions sont stockées dans une table afin d'éviter de résoudre le même sous problème plusieurs fois.

I. Utiliser la formule :$C_n^k = C_{n-1}^k + C_{n-1}^{k-1}$ pour calculer C_n^k itérativement en utilisant une table contenant les valeurs de C_i^j pour i = 0, n et j = 0,k. Calcul du nième terme de la suite de Fibonacci itérativement en utilisant une table contenant les valeurs de Fib(i) pour i=0,n.

II. Calcul du nième terme de la suite de Fibonacci itérativement en utilisant une table contenant les valeurs de Fib(i) pour i=0,n.

- Méthode par séparation et évaluation : un cas particulier du 'diviser pour régner' où les sous problèmes sont disjoints formant une arborescence où on doit encadrer la solution d'un sous problème par deux bornes inférieure et supérieure, un bon choix de ces derniers permettra d'abandonner plusieurs branches afin d'éviter l'exploration exhaustive et accélérer l'algorithme.

3.3 Exemples

Pour calculer le plus court cycle hamiltonien en partant du sommet a, on teste tous les successeurs de a et on estime pour chaque cas la longueur maximale (borne supérieure) et minimale (borne inférieure) du cycle ; trois situations sont alors possibles :

I. Si les deux bornes sont égales, la solution optimale est trouvée et on arrête l'exploration.

II. Si la borne inférieure est strictement supérieure à la borne supérieure, on abandonne cette branche puisqu'elle est non réalisable.

III. Si la borne inférieure est strictement inférieure à la borne supérieure, on refait le même procédé avec les successeurs de ce noeud (on explore la branche.)

Remark 3.4 *Cette méthode s'appuie sur les heuristiques de calcul des deux bornes; plus les deux bornes sont proches, plus l'algorithme converge rapidement et vice-versa.*

- Algorithme glouton : lorsque la solution est construite par étapes (ce qui est la plupart des cas), la meilleure décision (optimale) est prise à chaque étape localement sans tenir compte ni du passé ni du futur du processus de décisions dans l'espoir d'aboutir à une bonne solution.

ISTA

Exemples:

- Pour calculer le plus court chemin de a à b, on prend le sommet le plus proche de a, soit c, puis le plus proche de c, soit d, puis le proche de d et ainsi de suite sans tenir compte de la longueur cumulée par ces choix.
- Pour rendre le minimum de pièces de monnaie, on commence par la pièce de plus grande valeur possible et ainsi de suite.

Remarque L'algorithme glouton pourrait fournir une solution exacte (optimale) dans certains cas

3.4 Complexité des algorithmes / des problèmes

On a vu dans la section précèdente qu'un programme est de structures de contrôle (les instructions) manipulant des structures de données (variables et constantes).

Pour exécuter votre programme, vous avez donc besoin d'espace mémoire pour stocker les SDD et un processeur pour exécuter les SDC. L'ordinateur dispose de deux ressources nécessaires pour exécuter un programme : espace mémoire et temps processeur (architecture de Von Neumann RAM-CPU) ; La complexité d'un algorithme est une mesure de la quantité de ces ressources consommée par l'exécution de cet algorithme;

On distingue alors :

- La complexité spatiale : espace mémoire,$\cdots$;
- La complexité temporelle : temps,$\cdots$;

  $\cdots$ requis pour l'exécution de cet algorithme.

La première n'importe pas beaucoup avec l'évolution spectaculaire du hardware ; La seconde, quant à elle, est primordiale et dite alors complexité tout court (par abus de langage) .

Il est clair que le temps d'exécution d'un algorithme est fonction de ses opérations (plus particulièrement ses opérations fondamentales) et de la taille de son input (la quantité de données que prend l'algorithme en entrée) ;

Definition 3.2
- *La complexité d'un algorithme est le nombre d'opérations fondamentales que fait cet algorithme en fonction de la taille de son input.*
- *L'opération fondamentale est celle qui affecte le plus le temps d'exécution.*

Examples:

ISTA

i. *C'est la multiplication dans le calcul de n!.*

ii. *C'est l'addiction dans le calcul de Fib(n).*

iii. *C'est la comparaison dans la recherche d'un élément dans un tableau.*

- *La complexité est donc une estimation formelle (mathématique) du temps d'exécution de l'algorithme en fonction de la taille de son input.*
- *Elle est indépendante de:*

∗ *De la machine,· · · ;*

∗ *De l'OS,· · · ;*

∗ *Du langage · · · à utiliser pour exécuter l'algorithme.*

On distingue, suivant l'instance de l'algorithme de données, la complexité:

- *Dans les pires cas ;*
- *Dans les meilleurs cas ;*
- *Dans le cas moyen ;*

Par exemple,lors de la recherche de votre nome dans une liste de n noms, on fera :

- *Une seule comparaison, dans le meilleur cas, si votre nom est en tête de liste ;*
- *n comparaisons, dans le pire cas, si votre nom est le dernier de la liste ;*
- $\frac{\sum_{i=i}^{M}}{M} r_i$ *comparaisons (M est le nombre d'instances,* r_i *est le rang de votre nom dans l'instance i) dans le moyen cas.*

Remarquons que la complexité dans les moyens cas est souvent difficile à évaluer puisque le nombre d'instances est dans la plupart des problèmes infini.

Une définition plus formelle

La complexité est le nombre d'opérations élémentaires qu'effectue une machine de Turing pour reconnaitre un mot en fonction de la longueur de ce mot. Elle sert à analyser, évaluer, comparer les algorithmes.

Ce qui importe c'est quand la taille de l'input est assez grande : allure asymptotique du nombre d'opérations.

ISTA

Exemple: Calcul de x^n

***Premier algorithme**: en utilisant la définition : $x^n = x.x.x\cdots$, nfois : Float power(int n) $\{p = 1; for(j = 0; j < n-1; j++)p^* = x; return\ p;\}$ L'opération fondamentale est la multiplication dont le nombre est $T(n) = n-1$. On peut en gagner une en introduisant : Float power(int n) $\{p = x; for(j = 0; j < n-2; j++)p^* = x; returnp;\}$ et on aura $T(n) = n-2$. Dans les deux cas, quand n est assez grand, $T(n) \approx n$ (plus précisément T(n) est majoré par c.n),i.e $\lim_{n\to\infty} \frac{T(n)}{n} = 0$.*

***Deuxième algorithme**: en utilisant la formule: $x^n = x^2.x^2.x^2.\cdots.x^2$ si n est pair et $x^n = x^2.x^2.x^2.\cdots.x$ si n est impair.On aura $OnauraT(n) \approx n/2$ (et on pourrait faire mieux).*

3.5 Calcul de la complexité

Soit P1 et P2 deux programmes et $T(x)$ le nombre d'opérations fondamentales que fait l'algorithme x:

- T(x=a) = 1 (une unité)
- T(if (condition) P1) = T(P1)
- T(if (condition) P1 else P2) = max (T(P1) , T(P2)
- T(for (i=0 ; i<k ; i++) P1) = k.T(P1)
- T(while (condition) P1) = max iterations.T(P1)
- T(P1; P2)= T(P1) + T(P2)
- T(P1 // P2)= max (T(P1) , T(P2))

Si le programme est itératif et n est la taille de son input : T(n) est une fonction de n .

Exemples: $T(n) = 3n+2; T(n) = log(n); T(n) = n^2+n-1; T(n) = 2^n, \cdots$

Remarque: Si le programme est récursif et n est la taille de l'input : T(n) est récurrente et est une fonction de T(n-1) ou T(n/2) ou T(n-2), ...; avec un cas trivial comme T(0)= 1 et/ou T(1)= 1 , ...

Exemples:

- $T(1) = 1$ et $T(n) = T(n-1) + 2 \to T(n) = 2n+1$;
- $T(1) = 0$ et $T(n) = T(n-1) + n \to T(n) = n(n-1) = n^2 - n$;
- $T(1) = 1$ et $T(n) = 2 * T(n/2) + 1 \to T(n) = log_2 n$;

ISTA

3.6 Notation de Landau

Le mathématicien allemand Edmund Landau s'est intéressé à l'étude asymptotique des fonctions numériques à variable entière en définissant les notations suivantes:

- $f = O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}, \forall n \geq n_0 : f(n) \leq c.g(n)$
- $f = \Omega(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}, \forall n \geq n_0 : f(n) \geq c.g(n)$
- $f = \Theta(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c_1, c_1 \in \mathbb{R}, \forall n \geq n_0 : c_1.g(n) \leq f(n) \leq c_1.g(n)$ La notation la plus utilisée est O (se lit big o). On écrit :$f = O(g(n))$ ou $f \in O(g(n))$ et on dit que l'algorithme est en $O(g(n))$. Ceci veut dire : Si n est assez grand, f(n) est de l'ordre de g(n) ; Ces notations permettent de décrire l'allure la courbe représentative de la fonction f quand n est assez grand $(n \to \infty)$.

Si $T(n) = 3n + 2$ alors $T(n) = O(n)$ puisque on peut aisément trouver un réel c et un entier n_0 tels que $T(n) \leq c.n$ pour tout $n \geq n_0$.

En effet : $T(n) \leq c.n \Leftrightarrow 3n+2 \leq c.n \Leftrightarrow n \geq \frac{2}{c-3}$. En prenant par exemple c = 4 et $n_0 = 2$ on trouve que $\exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}, n \geq n_0 : T(n) \leq c.n$ c.-à-d. $T(n) = O(n)$.

Remarques

- $f = O(g) \Leftrightarrow g = \Omega(f)$
- $f = \Theta(g) \Leftrightarrow f = O(g)$ and $g = O(f)$
- Θ est symétrique: $f = \Theta(g) \Leftrightarrow g = \Theta(f)$
- O, Ω sont antisymétriques.
- O, Θ, Ω sont réflexives et transitives.

3.7 Complexités usuelles

3.8 Classes de problème

Un problème de décision est un problème dont la solution est 'vrai' ou 'faux' (oui ou non).

Exemples

- L'élément x est-il dans le tableau $A[]$?

T(n)	Notation en O	Dénomination
Constante (1, 10, 500,..)	O(1)	Constante (Polynomiale)
an+b	O(n)	Linéaire (Polynomiale)
aLog n+b	O(logn)	Logarithmique (Polynomiale)
an^2+bn+c	$O(n^2)$	Quadratique (Polynomiale)
anlogn+b	O(nlogn)	Quasi logarithmique (Polynomiale)
Polynôme en n degré k	$O(n^k)$	Polynomiale
$c.a^n$ (a >1)	$O(a^n) \approx O(2^n)$	Exponentielle
c.n !	$O(n!) \approx O(2^n)$	Exponentielle

(a, b, c des constantes)

Exemples

Comparer les algorithmes de tri de nombres

i. **Enumération** (idiot) nombre d'opérations borné par c.n!
ii. **Sélection** (bête) nombre d'opérations borné par $a.n.(n-1)/2 = O(n^2)$
iii. **Rapide** (malin) nombre d'opérations borné par a.n.logn = O(nlogn)

Si une opération se déroule en 10^{-6}seconde, le nombre d'opérations est de l'ordre de :

n	Log n	n	n^2	2^n	n!
10	2.3×10^{-6} s	10^{-5} s	10^{-4} s	10^{-3} s	3.6 s
20	3.2×10^{-6} s	2×10^{-5} s	4×10^{-4} s	33.55 s	491×10^{9} a
50	3.9×10^{-6} s	5×10^{-5} s	25×10^{-4} s	35.7 a	-
	Complexités polynomiales			Complexités exponentielles	

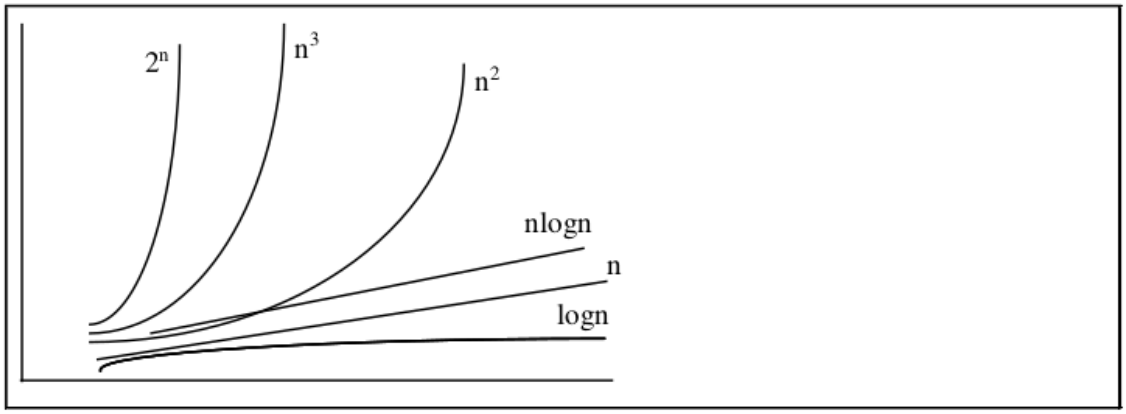


Figure 3.1: Allure de quelques courbes

- Le tableau $A[]$ est-il trié par ordre croissant ?
- x est-il le max dans le tableau $A[]$?
- n est-il premier ? Le problème de primalité (Prime Problem, PP).
- Peut-on colorer les sommets d'un graphe par 3 couleurs tels que deux sommets adjacents aient 2 couleurs différentes ? Problème de k-Coloration (k-Coloring Problem).
- . Existe-il un cycle hamiltonien de longueur l dans le graphe G ?
- . Peut-on placer 8 reines sur un échiquier sans qu'elles se menacent ?

Définition: La complexité d'un problème est celle du meilleur algorithme le résolvant. Meilleur = le plus rapide (le plus efficace).

3.9 Classification des problèmes de décision:P, NP, NP-complet, NP-difficile.

Classes P

On dit qu'un problème de décision est dans la classe P (Polynomial time) ou « facile, easy » si et seulement s'il existe un algorithme polynomial déterministe le résolvant.

Exemples

- L'élément x est-il dans le tableau A[] ?
- Le tableau A[] est-il tiré par ordre croissant ?
- Existe-il un cycle eulérien de longueur minimale dans le graphe G ?

ISTA

Classes NP

On dit qu'un problème est dans la classe NP (Non deterministic Polynomial time) si et seulement s'il existe un algorithme polynomial non déterministe le résolvant , c.-à-d., connaissant une solution, elle peut être vérifiée par un algorithme polynomial déterministe. Le seul algorithme déterministe pouvant résoudre ce problème est exponentiel.

Exemples

- Peut-on colorer les sommets d'un graphe par 3 couleurs tels que deux sommets adjacents aient 2 couleurs différentes ?
- Existe-il un cycle hamiltonien de longueur minimale dans le graphe G ?
- Peut-on placer 8 reines sur un échiquier sans qu'elles se menacent ?

Réduction polynomiale: On que le problème A est une réduction polynomiale du problème B (ou A se réduit polynomialement à B) si et seulement si A peut être résolu en un temps polynomial en fonction du nombre d'appels de B et on écrit: $A \leq_P B$.

Problème NP-difficile

Un problème X est dit NP-difficile si et seulement si tout problème de NP peut être réduit polynomialement à X.
$A \in NP - Hssi \exists B \in NP$ tel que $A \leq_P B$ (NP-Difficile en anglaise NP-Hard ou NP-H).
$(NP - H) \cap NP = NP - C$ et $NP - C \subseteq NP$ Il est clair que $P \subset NP$ mais est-ce-que $NP \subset P$?
$P = NP$? Problème ouvert millénaire 1.000.000 dollars !

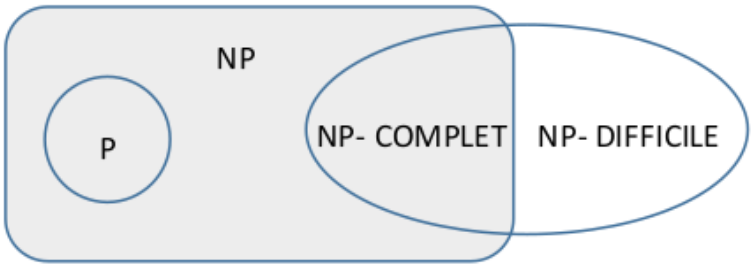


Figure 3.2: Classes de problèmes.

Pour montrer qu'un problème A est polynomial, il faut trouver un algorithme pour le résoudre et prouver que cet algorithme s'exécute en un temps qui augmente de façon polynomiale en fonction de la taille de l'instance traitée. Pour montrer qu'un problème A est NP-complet, on choisit un problème déjà connu pour être NP- complet, soit A_{nc} , et on montre que A_{nc} peut se "transformer" en A.

Donc, si on savait résoudre A, on saurait résoudre A_{nc} (A est aussi difficile que A_{nc}). Or, on ne sait pas résoudre P_{nc} : donc il va sûrement être difficile de résoudre P. A va, à son tour, être classé $NP - complet$.

Si on savait résoudre facilement A on saurait résoudre aussi A_{nc} ; Or on ne sait pas résoudre A_{nc} , A est donc sûrement difficile à résoudre.

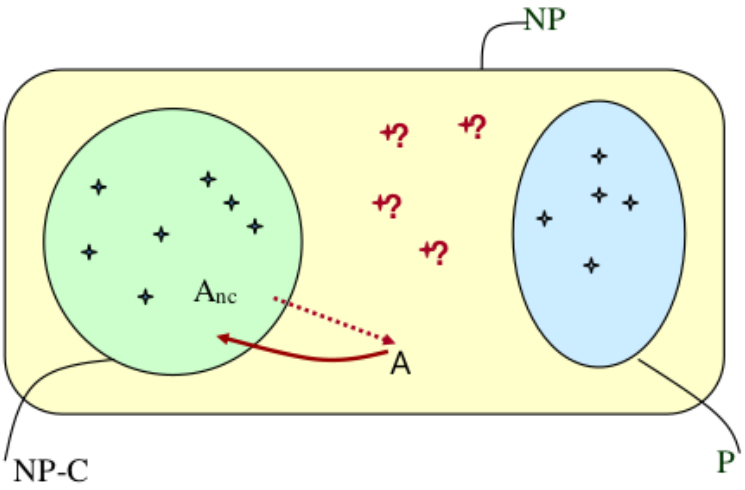


Figure 3.3: Classes de problèmes.

Le problème SAT ("satisfiabilité" d'une expression logique) Etant donne une fonction booléenne f à n variables booléennes $x_i, i = 1, \cdots, n$. Existe-t-il un vecteur x de $\{0,1\}^n$ pour lequel $f(x) = 1$ (true)?

Exemple

$f(x,y,z,t) = (x \vee y \vee z) \wedge (x \vee y \vee \bar{t}) \wedge (y \vee t) \wedge (\bar{x} \vee z)$ Peut-on affecter des valeurs vrai ou faux aux variables de telle façon que l'expression soit vraie ?Exercice Une solution : x=vrai, y=faux, z=vrai , t=vrai.

Théorème 3.1 *Le théorème de Cook: SAT est le premier problème NP-complet connu.*

ISTA

Références

1. Approche de la théorie de la complexité, Théorie développée à la fin du 20ème siècle (S. Cook.
2. Optimisation Combinatoire, Gérard Verfaillie ;ONERA/DCSD/CD, Toulouse ,2008.
3. Optimisation Combinatoire : Programmation Linéaire et Algorithmes,Pierre Fouilhoux;2015.
4. Combinatorial Optimization: Theory and Algorithms,Fifth Edition Bernhard Korte and Jens Vygen.

ISTA